PROJECTOR-RESOLVED SEED CALCULUS

Universal Unknown-Edge Recovery

Impossibility without structure, exact off-grid recovery in a finite-rate class, and Gibbs-free synthesis

A truly unrestricted recovery theorem cannot exist. This book proves that obstruction first. It then gives a complete recovery theorem for every periodic signal consisting of an unknown bandlimited background and finitely many unknown off-grid jumps. The number of jumps, their locations, their amplitudes, the edge packet, and the smooth packet are all reconstructed from finitely many Fourier coefficients.

Komorebi P.

Abstract

Unknown-edge recovery is not meaningful until the measurement map, background class, edge class, and stability scale are declared. If arbitrary functions are allowed, a nonzero interval can lie between every pair of sample points and remain invisible. If the smooth and edge classes overlap, the decomposition is not unique. If two opposite jumps collide, their Fourier moments approach zero, so no recovery map can have a uniform stability constant over all edge separations.

After proving those limits, this work gives two positive theorems. The first is finite-dimensional. Let $\mathcal{S}$ be a background subspace, let D be an edge dictionary, and let $R_{\mathcal{S}} = I - P_{\mathcal{S}}$. Every edge packet with at most s active atoms is recoverable from $y = s_0 + D\alpha$ if and only if

$$\operatorname{Ker}(R_{\mathcal{S}} D) \cap \{h : \|h\|_0 \le 2s\} = \{0\}.$$

The associated restricted separation constant gives an explicit deterministic noise bound.

The second theorem is continuous and off-grid. Let

$$f = s + j$$

be a real 2π-periodic signal. Assume s is an unknown trigonometric polynomial of degree at most K. Assume j is an unknown piecewise-constant function with at most L distinct jumps. For frequencies $n > K$,

$$m_n = 2\pi \mathrm{i} n \widehat{f}(n) = \sum_{\ell=1}^{L} a_\ell z_\ell^n, \qquad z_\ell = \mathrm{e}^{-\mathrm{i}\tau_\ell}.$$

The edge moments form a finite-rank Hankel packet. Its rank is the unknown number of jumps. Its null recurrence gives an annihilating polynomial whose roots are the unknown edge locations. A Vandermonde solve gives the jump amplitudes. The edge packet follows by periodic integration. The low Fourier modes then recover the smooth packet. The reconstructed signal is exact throughout the declared class.

The same recovery removes the Gibbs mechanism from the final synthesis. The edge is not approximated by a truncated global Fourier series. It is recovered and restored in its own packet. The supplied single Python file generates thirty-five figure pairs, five data tables, four exact random examples, a symbolic roots-of-unity derivation, a two-dimensional rectangle example, noise and collision experiments, and nineteen independent checks.

The result in one page

What cannot be true. No method can recover arbitrary unknown edges from arbitrary finite measurements. No stable theorem can be uniform over colliding edges. A claim of universality without a model class is false.

What is proved. Every signal in the class $\mathcal{U}_{K,L}$, consisting of a degree-K periodic trigonometric background plus at most L distinct piecewise-constant jumps, is recovered exactly from finitely many Fourier coefficients. The true value of L may be unknown as long as an upper bound is supplied.

What the calculus adds. The smooth component is removed by a declared projector. The discontinuity becomes a finite derivative packet. The Hankel matrix is a transfer packet with exact rank L. The annihilator is the obstruction-free recurrence. The edge and smooth packets are reconstructed separately and then synthesized exactly.

The core formula is

$$\boxed{2\pi i n \widehat{f}(n) = \sum_{\ell=1}^{L} a_\ell \mathrm{e}^{-\mathrm{i}n\tau_\ell} \qquad (n > K).}$$

From $2L$ consecutive values of this sequence, the edge locations and amplitudes are determined by finite linear algebra. If only an upper bound $L_{\max}$ is known, the rank of an $L_{\max} \times L_{\max}$ Hankel matrix gives the actual number of active edges.

Claim boundary

The word *universal* is used in the ordinary theorem sense. One algorithm works for every member of one declared class. It does not mean every possible edge in every possible function can be recovered from every kind of data.

The following statements are established here.

- Unrestricted universal recovery from noninjective measurements is impossible.
- Universal sparse recovery in a finite edge dictionary has a necessary and sufficient secant condition.
- Periodic off-grid piecewise-constant edges mixed with an unknown bandlimited background are exactly recoverable from finitely many Fourier coefficients.
- The unknown number of jumps is the Hankel rank under the stated nondegeneracy assumptions.
- Shared edge locations across several channels have a common annihilator.
- Piecewise-polynomial packets reduce to a repeated-root annihilator.
- Exact edge recovery followed by native packet synthesis removes jump-generated Gibbs ringing from the reconstructed signal.

The following statements are not established.

- Exact recovery of arbitrary sub-grid edges from arbitrary point samples.
- Uniform noise stability without a lower bound on edge separation or an equivalent conditioning hypothesis.
- Exact recovery of arbitrary two-dimensional curves from finitely many unrestricted measurements.
- A claim that annihilating filters, Prony reconstruction, finite-rate sampling, or super-resolution were invented here.

Reading path

The proof has four layers.

1. Chapters 1–3 show why a model-free theorem cannot exist.
2. Chapters 4–7 prove the finite dictionary theorem and its stability estimate.
3. Chapters 8–17 prove the off-grid finite-rate theorem from the distributional derivative through final reconstruction.
4. Chapters 18–27 study Gibbs synthesis, noise, edge collision, multiple channels, piecewise polynomials, and implementation.

Contents

I The boundary of universality **1**

1 What unknown-edge recovery means **1**
1.1 The object to be recovered . . . 1
1.2 Three meanings of universal . . . 1

2 Why model-free recovery is impossible **2**
2.1 The measurement obstruction . . . 2
2.2 The decomposition obstruction . . . 2

3 Why no uniform noise bound can ignore edge separation **3**
3.1 A collapsing pair of jumps . . . 3

II A universal theorem for finite edge dictionaries **4**

4 Removing the background packet **4**
4.1 Finite-dimensional model . . . 4
4.2 Why pairwise differences control recovery . . . 5

5 The residualized secant theorem **5**

6 A deterministic stability theorem **6**
6.1 Restricted separation constant . . . 6
6.2 What the constant means . . . 6

7 Worked finite dictionary examples **7**
7.1 One-dimensional two-atom packet . . . 7
7.2 A two-dimensional rectangle packet . . . 8
7.3 Why this theorem matters for the larger problem . . . 8

III Exact off-grid recovery from Fourier packets **8**

8 The finite-rate edge class **8**
8.1 Periodic coordinate and Fourier convention . . . 8
8.2 The declared class . . . 9
8.3 Why the class is not small in the relevant sense . . . 10

9 The derivative packet **10**
9.1 Distributional derivative . . . 10
9.2 Fourier transform of the derivative packet . . . 11

10 Removing the unknown smooth background **11**
10.1 High-frequency packet separation . . . 11
10.2 The operator interpretation . . . 12

11 The Hankel packet and its rank **12**
11.1 Hankel construction . . . 12
11.2 Unknown number of edges . . . 13

12 The annihilating polynomial **14**
12.1 Construction . . . 14
12.2 Finite linear system . . . 15

13 Recovering jump amplitudes and the edge packet **15**
13.1 Vandermonde solve . . . 15
13.2 Periodic integration . . . 16

14 Recovering the smooth packet **17**

15 The universal off-grid recovery theorem **17**
15.1 Representative numerical values . . . 18

IV Gibbs-free synthesis after unknown-edge recovery **19**

16 Why exact edge recovery changes the reconstruction problem **19**
16.1 The raw Fourier route . . . 19
16.2 The resolved synthesis route . . . 20
16.3 Error decomposition when the edge is only approximate . . . 20

17 The closed recovery algorithm **21**

V Stability, conditioning, and the exact boundary **22**

18 Why exact recovery is not the same as uniform stability **22**
18.1 Fourier noise becomes derivative-moment noise . . . 22
18.2 Hankel perturbation . . . 22
18.3 Root localization by Rouché's theorem . . . 22
18.4 Amplitude stability . . . 24

19 Collision is the unavoidable hard boundary **24**

20 Finite noise experiments **24**

VI Extensions of the same recovery law **25**

21 Several channels with shared edge locations **25**

22 Piecewise-polynomial packets **26**

23 Two-dimensional finite dictionaries **27**

VII Failure modes, complexity, and implementation **28**

24 Two ways the exact theorem can be used incorrectly **28**
24.1 Too few moments . . . 28
24.2 An invalid background bandlimit . . . 28
24.3 Model mismatch . . . 29

25 Computational cost **29**

26 A symbolic four-edge calculation **30**

27 Verification ledger **31**

28 How the pieces depend on one another **32**

VIII Relation to established finite-rate recovery **34**

29 What is classical and what is organized here **34**

30 The strongest defensible conclusion **34**

A Detailed exact-recovery table **34**

B Notation and implementation map **35**

C Complete single-file implementation **35**

D Bibliographic note **61**

E Reproduction **61**

Figures

1.1 The complete recovery chain used in this book. 1
4.1 The residualized secant condition. 4
7.1 Exact recovery in a finite one-dimensional dictionary. 7
7.2 A dimension-independent finite-dictionary recovery. 8
8.1 One representative member of the finite-rate class. 10
9.1 The edge packet becomes a finite atomic derivative. 11
11.1 A representative Hankel packet. 13
11.2 The unknown number of edges appears as a rank break. 14
13.1 Exact jump-amplitude recovery. 16
14.1 Recovered smooth, edge, and complete packets. 17
15.2 The final reconstruction residual. 19
18.1 Certified root neighborhoods. 23
21.1 Two channels with one shared edge geometry. 26
22.1 From jumps to higher-order innovations. . . . 27
24.1 The moment-count boundary. 28
24.2 The declared bandlimit matters. 29
25.1 Measured direct runtime. 30
27.1 The complete finite verification dashboard. . . 32
28.1 Proof dependency map. 32
28.2 What is proved and what remains outside the theorem. 33
28.3 The unknown-edge recovery wheel. 33

Part I: The boundary of universality

1 What unknown-edge recovery means

1.1 The object to be recovered

An edge is not a number attached to a plot. It is a failure of one local description to continue across a boundary. In one dimension a piecewise-constant edge packet has a distributional derivative supported on finitely many points. In a sampled image an edge packet may be a member of a finite geometric dictionary. In a graph signal it may be a sparse gradient across selected graph edges.

The recovery problem must therefore declare four objects.

1. A measurement map M.
2. A background class $\mathcal{S}$.
3. An edge class $\mathcal{J}$.
4. A rule for deciding when two edge packets are distinguishable.

Given data

$$y = M(s + j) + \eta, \qquad s \in \mathcal{S}, \qquad j \in \mathcal{J},$$

unknown-edge recovery asks for j, its boundary parameters, and a reconstruction of s. Exact recovery means that the returned packets synthesize the original object, not merely that an image looks sharp.

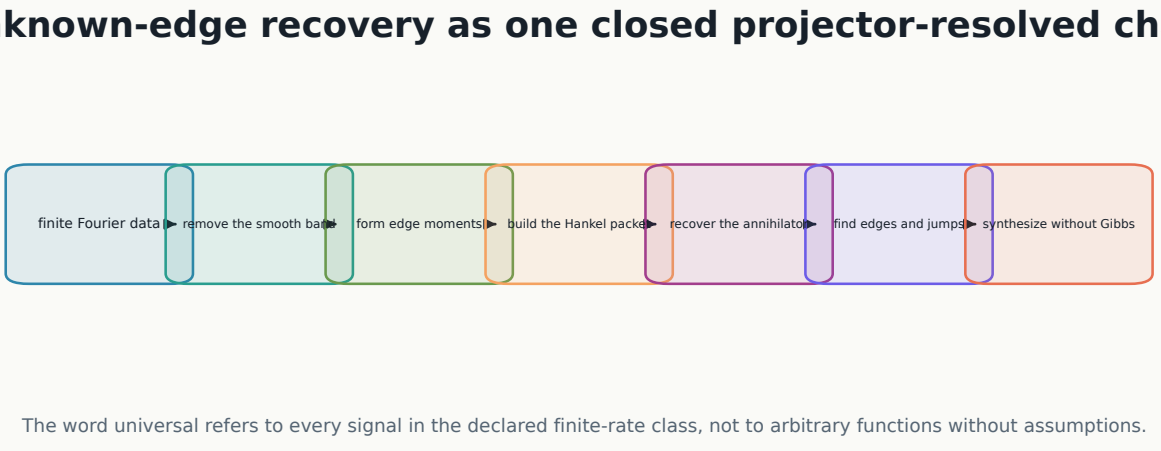


Figure 1.1: The complete recovery chain used in this book.

Reading the figure. The data are first separated from the smooth band. The remaining moments form a finite-rank packet. The annihilating recurrence produces edge locations. Native edge synthesis then avoids the Gibbs mechanism.

1.2 Three meanings of universal

The same word is often used for three different goals.

Model-free universality would recover edges in every admissible function from the stated measurements. This fails as soon as the measurement map is not injective.

Class universality means one algorithm works for every member of a declared class. This is the meaning proved here.

Uniform stability asks for one error constant valid across the class. This requires quantitative separation. Exact identifiability alone does not provide a useful noise constant when edges nearly collide.

Keeping these meanings separate is essential. The exact theorem is strong because its assumptions and conclusion match. It would become weaker, not stronger, if the assumptions were hidden behind an unrestricted use of the word universal.

2 Why model-free recovery is impossible

2.1 The measurement obstruction

Theorem 2.1 (Noninjective measurement obstruction)**.** *Let $M : X \to Y$ be a measurement map. Suppose there are two admissible objects $f_0, f_1 \in X$ with*

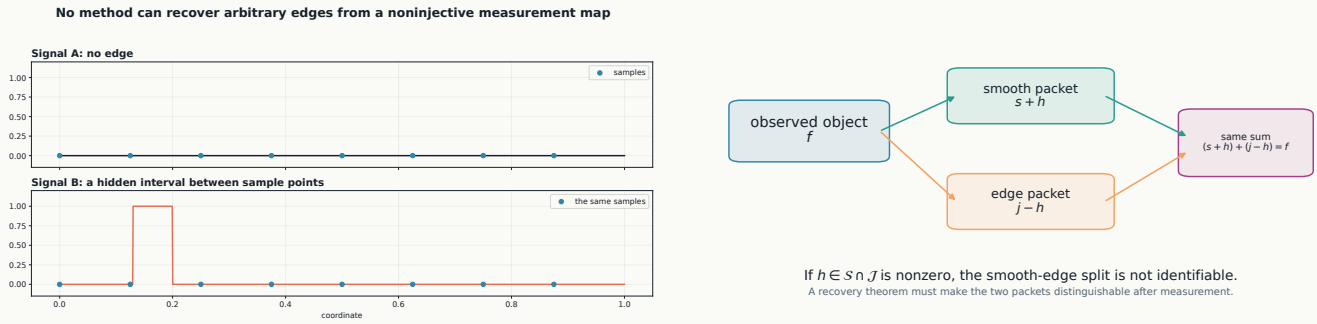
$$Mf_0 = Mf_1$$

and different edge packets. No recovery map $R : Y \to \mathcal{J}$ can return the correct edge packet for both objects.

Proof. The two objects produce the same datum $y = Mf_0 = Mf_1$. The map R has one value at y. That value cannot equal two different edge packets. □

Corollary 2.2 (Finite point samples do not see every edge)**.** *Fix finitely many sample points in an interval. There is a nonzero indicator packet supported between two adjacent sample points that vanishes at every sample point. Hence arbitrary unknown edges cannot be recovered from those samples.*

Proof. Choose a nonempty open interval containing no sample point. Its indicator is nonzero, has two edges, and is zero at every sampled coordinate. □



(a) A hidden interval between samples (b) A second source of nonuniqueness

Left. The two displayed signals have identical samples. Any deterministic algorithm receives exactly the same input for both. **Right.** Even full knowledge of the sum does not determine its smooth and edge components when the two classes overlap.

2.2 The decomposition obstruction

Theorem 2.3 (Intersection obstruction)**.** *Let $\mathcal{S}$ and $\mathcal{J}$ be linear subspaces of an ambient vector space. If*

$$\mathcal{S} \cap \mathcal{J} \neq \{0\},$$

then the decomposition $f = s + j$ is not unique.

Proof. Take a nonzero $h \in \mathcal{S} \cap \mathcal{J}$. Then

$$f = s + j = (s + h) + (j - h).$$

Both pairs belong to $\mathcal{S} \times \mathcal{J}$, and the pairs are different. □

The condition $\mathcal{S} \cap \mathcal{J} = \{0\}$ is necessary when the full object is known. Under an incomplete measurement map, one needs the stronger condition that no difference of two admissible edge packets becomes indistinguishable after the background has been removed.

3 Why no uniform noise bound can ignore edge separation

3.1 A collapsing pair of jumps

Consider the periodic interval packet

$$j_\delta = \mathbf{1}_{[0,\delta)}.$$

Its derivative packet is

$$j'_\delta = \delta_0 - \delta_\delta.$$

The corresponding Fourier moments are

$$m_n(\delta) = 1 - \mathrm{e}^{-in\delta}.$$

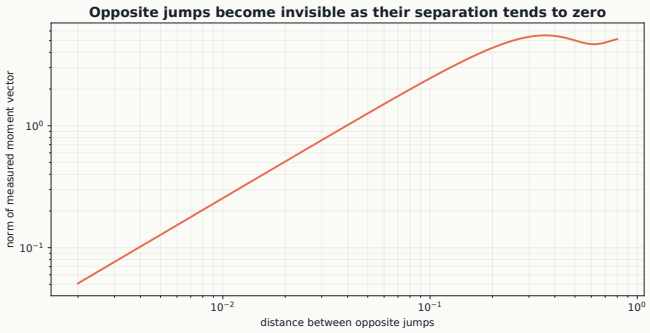
For every fixed finite set of indices,

$$\|m(\delta)\|_2 \longrightarrow 0 \qquad (\delta \to 0).$$

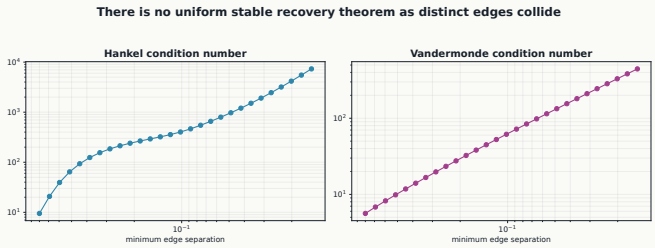
The edge amplitudes remain $+1$ and -1. Only their separation changes.

Theorem 3.1 (No uniform Lipschitz recovery over colliding edges)**.** *Let the measurement vector contain finitely many Fourier moments. There is no inverse map from moments to edge locations with one Lipschitz constant valid for all distinct edge configurations.*

Proof. The moment vector of the two-edge packet tends to the moment vector of the zero packet as $\delta \to 0$. The edge parameter does not converge to an ordinary two-edge parameter with fixed separation. Any uniform Lipschitz inverse would force a fixed nonzero edge displacement to be bounded by a constant times a measurement difference that tends to zero. □



(a) Moment collapse



(b) Conditioning near collision

Left. The measurement norm shrinks linearly at first as opposite jumps approach each other. **Right.** The Hankel and Vandermonde condition numbers rise as the minimum separation falls. Exact arithmetic still identifies distinct edges, but noise amplification becomes unavoidable.

This theorem does not weaken the exact result. It identifies the quantity that must appear in a stable result. One may state that quantity as minimum separation, a smallest Hankel singular value, a Vandermonde condition number, or a root-isolation margin. They express the same geometric fact from different coordinates.

Part II: A universal theorem for finite edge dictionaries

4 Removing the background packet

4.1 Finite-dimensional model

Let $X = \mathbb{R}^m$ with its usual inner product. Let $\mathcal{S} \subseteq X$ be a known background subspace. Let

$$D = \begin{pmatrix} d_1 & \cdots & d_N \end{pmatrix}$$

be a finite dictionary of candidate edge atoms. The observation has the form

$$y = s_0 + D\alpha_0 + \eta, \qquad s_0 \in \mathcal{S}.$$

The unknown coefficient vector α_0 is assumed to have at most s nonzero entries.

Let $P_{\mathcal{S}}$ be the orthogonal projector onto $\mathcal{S}$, and put

$$R_{\mathcal{S}} = I - P_{\mathcal{S}}.$$

Applying $R_{\mathcal{S}}$ removes every possible smooth coefficient at once:

$$R_{\mathcal{S}} y = R_{\mathcal{S}} D\alpha_0 + R_{\mathcal{S}}\eta.$$

The residualized dictionary

$$A = R_{\mathcal{S}} D$$

is the actual edge measurement operator. The background basis no longer appears in the search.

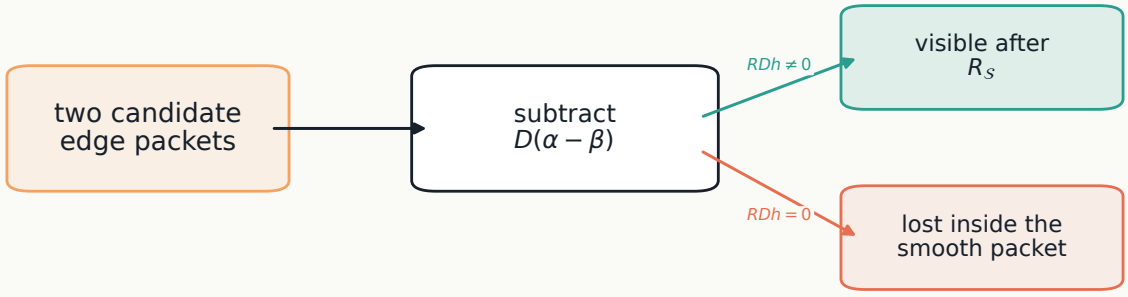


Figure 4.1: The residualized secant condition.

Reading the figure. Two candidate edge packets can be distinguished exactly when their difference remains visible after projection away from the background.

4.2 Why pairwise differences control recovery

Suppose two s-sparse vectors α and β explain the same residual data. Then

$$A\alpha = A\beta,$$

so

$$A(\alpha - \beta) = 0.$$

The difference $h = \alpha - \beta$ has at most $2s$ nonzero entries. Therefore the recovery problem is not governed by the individual atoms alone. It is governed by the complete set of $2s$-sparse secants.

5 The residualized secant theorem

Theorem 5.1 (Universal finite-dictionary recovery)**.** *The following statements are equivalent.*

1. *Every s-sparse edge coefficient vector is uniquely determined from $A\alpha$.*
2. $$\operatorname{Ker} A \cap \{h : \|h\|_0 \le 2s\} = \{0\}.$$
3. *Every set of at most $2s$ columns of A is linearly independent.*
4. $$\operatorname{spark}(A) > 2s.$$

When these conditions hold, the solution of

$$\boxed{\min_{\alpha} \|\alpha\|_0 \quad \textit{subject to} \quad A\alpha = R_{\mathcal{S}} y}$$

is the true edge coefficient vector for every noise-free observation in the class.

Proof. Assume statement 2. Let α and β be s-sparse and satisfy $A\alpha = A\beta$. Then $h = \alpha - \beta$ lies in $\operatorname{Ker} A$, and $\|h\|_0 \le 2s$. Statement 2 gives $h = 0$, hence $\alpha = \beta$.

Conversely, if statement 2 fails, choose a nonzero $h \in \operatorname{Ker} A$ with $\|h\|_0 \le 2s$. Split the support of h into two disjoint sets of size at most s. Write

$$h = \alpha - \beta$$

with α and β supported on those two sets. Then $A\alpha = A\beta$ even though $\alpha \neq \beta$. Universal uniqueness fails.

Statements 2, 3, and 4 are the standard equivalent definitions of the spark condition. The optimization claim follows because the true s-sparse vector is feasible and no other vector with at most s nonzero entries can be feasible. □

Remark 5.2. The theorem is exact and dimension-independent. The atoms may be one-dimensional intervals, image masks, graph cuts, curve fragments, or any other finite family. The theorem does not say the combinatorial search is always cheap. It says exactly when the inverse problem has one answer.

6 A deterministic stability theorem

6.1 Restricted separation constant

Assume the exact secant condition holds. Since the dictionary is finite, define

$$\boxed{\gamma_{2s} = \min_{\substack{h \neq 0 \\ norm h_0 \leq 2s}} \frac{\|Ah\|_2}{\|h\|_2}.}$$

The minimum is positive. It measures the smallest visible difference between two admissible edge descriptions.

Theorem 6.1 (Finite-dictionary noise bound)**.** *Suppose*

$$y = s_0 + D\alpha_0 + \eta, \qquad \|R_{\mathcal{S}}\eta\|_2 \leq \varepsilon, \qquad \|\alpha_0\|_0 \leq s.$$

Let $\widehat{\alpha}$ *satisfy*

$$\|\widehat{\alpha}\|_0 \leq s$$

and

$$\|A\widehat{\alpha} - R_{\mathcal{S}}y\|_2 \leq \varepsilon.$$

Then

$$\boxed{\|\widehat{\alpha} - \alpha_0\|_2 \leq \frac{2\varepsilon}{\gamma_{2s}}.}$$

Consequently,

$$\|D(\widehat{\alpha} - \alpha_0)\|_2 \leq \frac{2\,\|D\|\,\varepsilon}{\gamma_{2s}}.$$

Proof. Put $h = \widehat{\alpha} - \alpha_0$. Its support has size at most $2s$. By the triangle inequality,

$$\begin{aligned} \|Ah\|_2 &\leq \|A\widehat{\alpha} - R_{\mathcal{S}}y\|_2 + \|R_{\mathcal{S}}y - A\alpha_0\|_2 \\ &\leq 2\varepsilon. \end{aligned}$$

The definition of γ_{2s} gives

$$\gamma_{2s}\,\|h\|_2 \leq \|Ah\|_2\,.$$

Combining the two estimates proves the first bound. The packet bound follows by applying D. □

Corollary 6.2 (Support recovery by thresholding)**.** *If*

$$\min_{k \in \operatorname{supp}\alpha_0} |(\alpha_0)_k| > \frac{4\varepsilon}{\gamma_{2s}},$$

then thresholding $\widehat{\alpha}$ *at* $2\varepsilon/\gamma_{2s}$ *recovers the true support.*

Proof. Every coordinate error is at most the ℓ^2 error. True nonzero coefficients remain above the threshold, while true zero coefficients remain below it. □

6.2 What the constant means

The quantity γ_{2s} is a finite obstruction margin. If it is zero, two admissible packet descriptions become identical after background removal. If it is positive but small, the problem is identifiable yet sensitive. The theorem makes that distinction visible.

7 Worked finite dictionary examples

7.1 One-dimensional two-atom packet

The code constructs a length-24 signal. The background space is spanned by

$$1, \cos x, \sin x, \cos 2x, \sin 2x.$$

The edge dictionary contains eight short interval atoms. The true coefficient vector uses two atoms with amplitudes 1.2 and -0.7. Complete support search finds the exact pair.

The computed restricted separation constant is

$$\boxed{\gamma_4 = 0.5116895108313039.}$$

The recovered support is

$$\boxed{\{1, 5\}.}$$

The reconstruction residual is 1.21×10^{-15}.

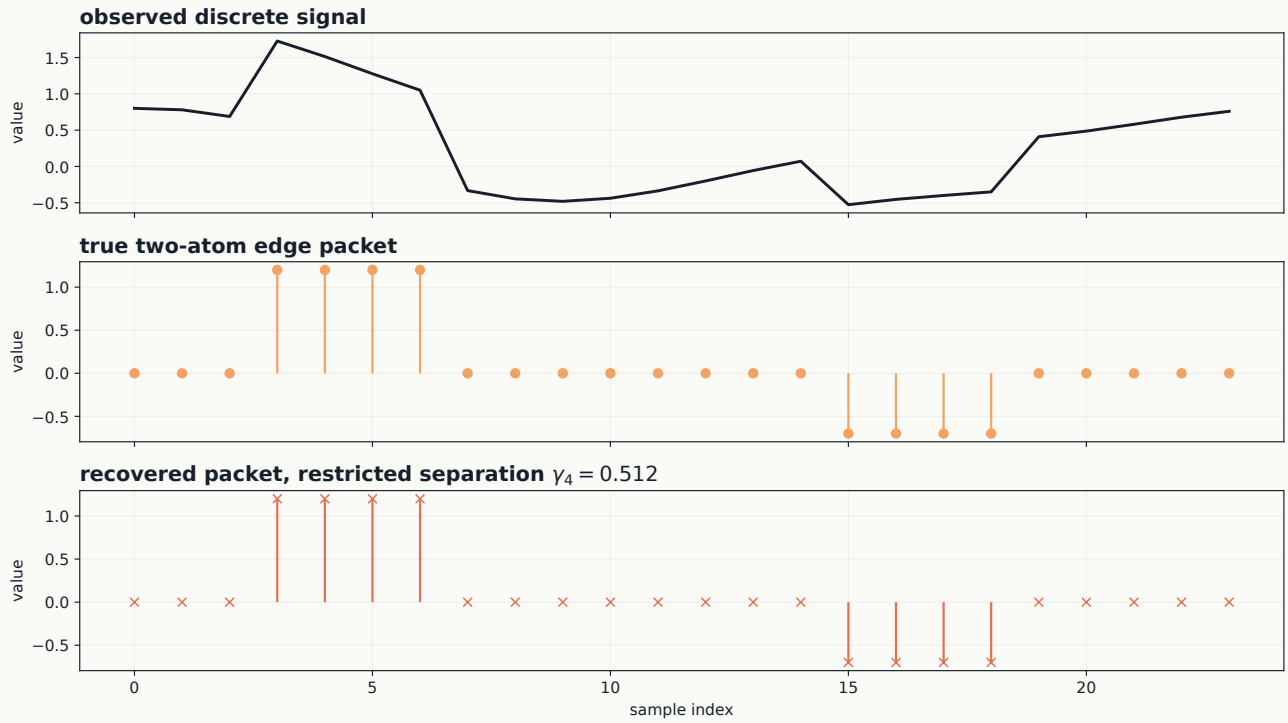


Figure 7.1: Exact recovery in a finite one-dimensional dictionary.

Reading the figure. The smooth background is removed before the sparse search. The two active interval atoms are recovered exactly. The positive value of γ_4 certifies uniqueness and controls the noisy coefficient error.

A deterministic noise vector of size 10^{-4} was then added. The measured coefficient error was

$$1.235 \times 10^{-4},$$

while the theorem gave the valid bound

$$2.078 \times 10^{-3}.$$

The bound is not intended to be sharp. Its role is to turn a visual recovery into a checkable inequality.

7.2 A two-dimensional rectangle packet

The same theorem is applied to an 18×18 image. The background is a six-dimensional smooth trigonometric space. The edge dictionary contains every nonwrapping axis-aligned rectangle with height and width between three and seven pixels.

The true rectangle is

$$(\text{top}, \text{left}, \text{height}, \text{width}) = (6, 8, 5, 6).$$

Profile least squares over the complete dictionary returns the same tuple and the correct amplitude 1.35. The largest reconstruction error is at floating-point scale.

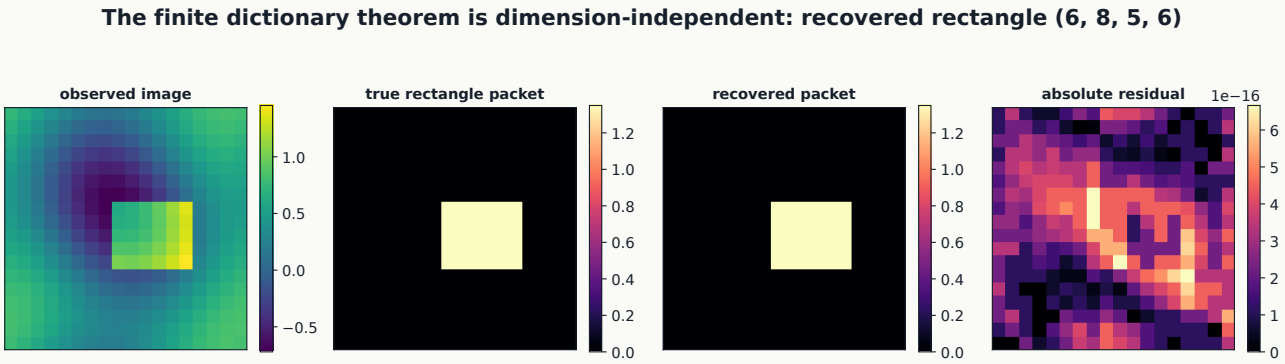


Figure 7.2: A dimension-independent finite-dictionary recovery.

Reading the figure. The observed image contains a smooth background and one unknown rectangle. The profile search recovers the exact rectangle and amplitude. The final panel shows only roundoff. This is not a theorem for arbitrary continuous curves. It is an exact theorem for the complete finite dictionary that was declared.

7.3 Why this theorem matters for the larger problem

The finite dictionary theorem already gives a universal answer for any finite packet library. Its weakness is computational and representational. A fine geometric library may be large, and an arbitrary off-grid edge is not one of its atoms. The next part removes that grid restriction in one dimension by changing coordinates from candidate masks to Fourier moments.

Part III: Exact off-grid recovery from Fourier packets

8 The finite-rate edge class

8.1 Periodic coordinate and Fourier convention

Work on

$$\mathbb{T} = \mathbb{R}/(2\pi\mathbb{Z}).$$

For an integrable periodic function u, define

$$\widehat{u}(n) = \frac{1}{2\pi} \int_0^{2\pi} u(x) \mathrm{e}^{-\mathrm{i}nx}\, dx, \qquad n \in \mathbb{Z}.$$

The inverse Fourier series is written

$$u(x) = \sum_{n\in\mathbb{Z}} \widehat{u}(n)\mathrm{e}^{\mathrm{i}nx}$$

whenever the usual convergence assumptions hold.

8.2 The declared class

Definition 8.1 (Universal finite-rate edge class)**.** *For nonnegative integers K and $L \geq 2$, let $\mathcal{U}_{K,L}$ consist of real periodic signals*

$$f = s + j$$

with the following properties.

1. *The background is a trigonometric polynomial*

$$s(x) = \sum_{|n|\leq K} c_n \mathrm{e}^{\mathrm{i}nx}, \qquad c_{-n} = \overline{c_n}.$$

2. *The edge packet j is piecewise constant with at most L distinct jumps at locations*

$$0 \leq \tau_1 < \cdots < \tau_r < 2\pi, \qquad r \leq L.$$

3. *The nonzero jump amplitudes are*

$$a_\ell = j(\tau_\ell^+) - j(\tau_\ell^-).$$

4. *Periodicity gives*

$$\sum_{\ell=1}^{r} a_\ell = 0.$$

5. *The edge packet is normalized to have mean zero. The missing constant is absorbed into s.*

The number r, the locations τ_ℓ, the jumps a_ℓ, and all background coefficients are unknown. Only the bounds K and L are assumed known.

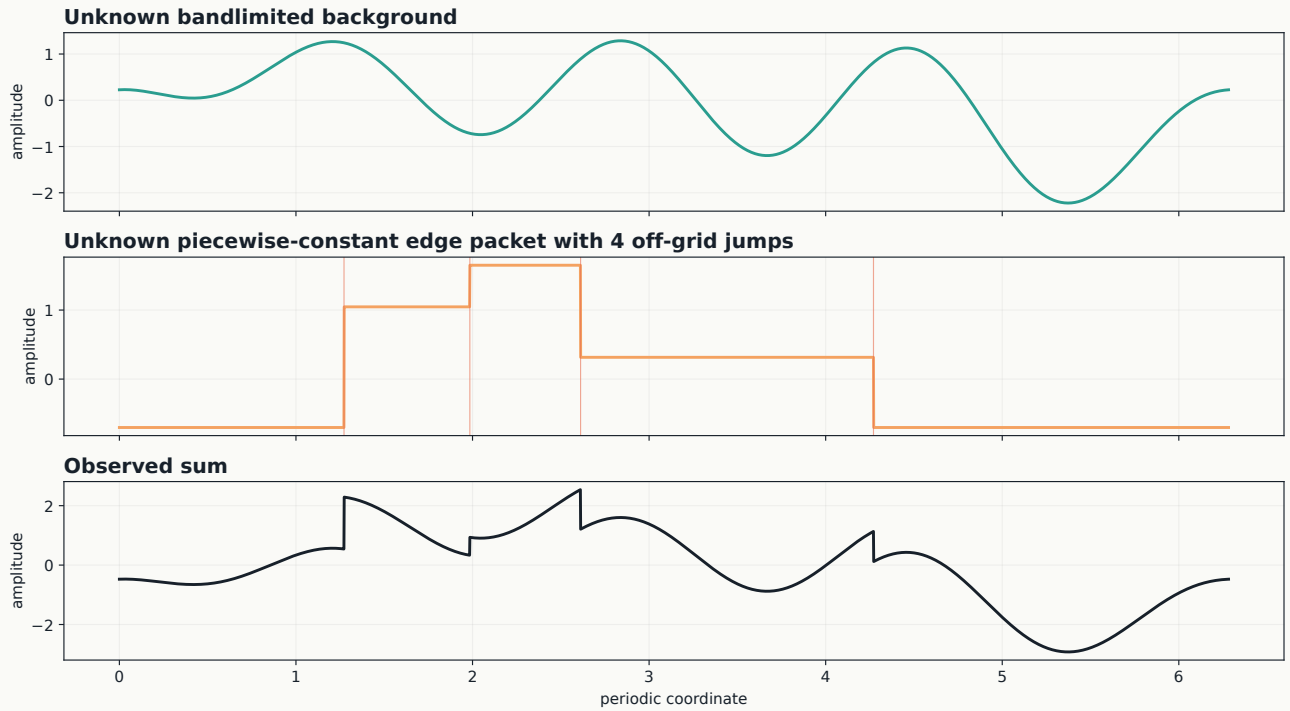


Figure 8.1: One representative member of the finite-rate class.

Reading the figure. The background, edge packet, and observed sum are all unknown to the recovery algorithm. The displayed case has four off-grid jumps and a degree-four background.

8.3 Why the class is not small in the relevant sense

The edge locations are continuous parameters. They are not restricted to a sample grid. The jump amplitudes are arbitrary nonzero real numbers satisfying one periodic balance equation. The background coefficients are arbitrary within the declared spectral band. The class includes intervals, multi-level staircases, irregular segment lengths, and any finite combination of upward and downward jumps.

At the same time, the class has a finite number of degrees of freedom. That finite-rate structure is what makes exact recovery from finitely many measurements possible.

9 The derivative packet

9.1 Distributional derivative

A piecewise-constant function has zero ordinary derivative inside each segment. All derivative mass is concentrated at the boundaries.

Proposition 9.1 (Edge derivative packet)**.** *For j in the declared class,*

$$\boxed{Dj = \sum_{\ell=1}^{r} a_\ell \delta_{\tau_\ell}}$$

in the sense of distributions.

Proof. Let φ be a smooth periodic test function. Integrating by parts on each constant segment gives

$$-\int_0^{2\pi} j(x)\varphi'(x)\,dx = \sum_{\ell=1}^{r} \big(j(\tau_\ell^+) - j(\tau_\ell^-)\big)\varphi(\tau_\ell).$$

The coefficient of $\varphi(\tau_\ell)$ is the jump amplitude a_ℓ. This is exactly the action of the stated sum of Dirac masses. □

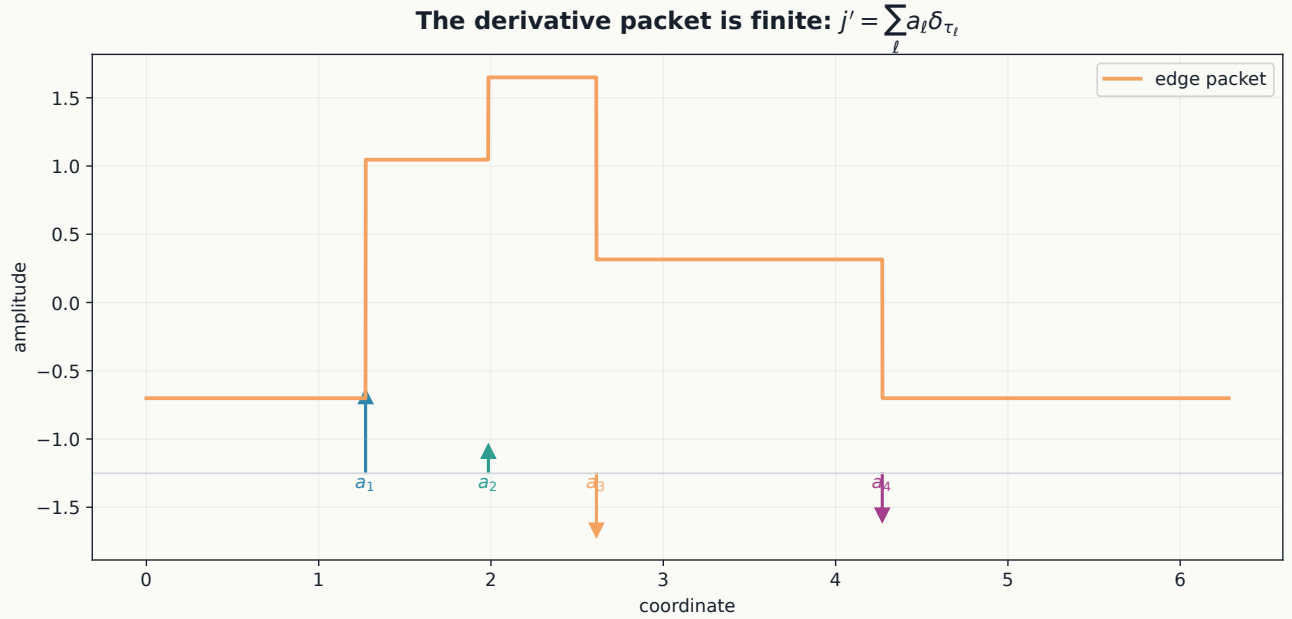


Figure 9.1: The edge packet becomes a finite atomic derivative.

Reading the figure. The piecewise-constant signal may occupy the entire domain, but its derivative is supported only at the unknown boundaries. This is the decisive compression step.

9.2 Fourier transform of the derivative packet

The Fourier coefficient of δ_τ is

$$\widehat{\delta_\tau}(n) = \frac{1}{2\pi} \mathrm{e}^{-\mathrm{i}n\tau}.$$

Differentiation multiplies Fourier coefficients by $\mathrm{i}n$. Therefore

$$\mathrm{i}n\widehat{j}(n) = \frac{1}{2\pi} \sum_{\ell=1}^{r} a_\ell \mathrm{e}^{-\mathrm{i}n\tau_\ell}.$$

For $n \neq 0$,

$$\boxed{2\pi \mathrm{i}n\widehat{j}(n) = \sum_{\ell=1}^{r} a_\ell z_\ell^n, \qquad z_\ell = \mathrm{e}^{-\mathrm{i}\tau_\ell}.}$$

Each unknown edge location is now a point on the unit circle. The moment sequence is a finite sum of exponentials.

10 Removing the unknown smooth background

10.1 High-frequency packet separation

Because s has degree at most K,

$$\widehat{s}(n) = 0 \qquad (|n| > K).$$

Hence for every positive $n > K$,

$$\widehat{f}(n) = \widehat{j}(n).$$

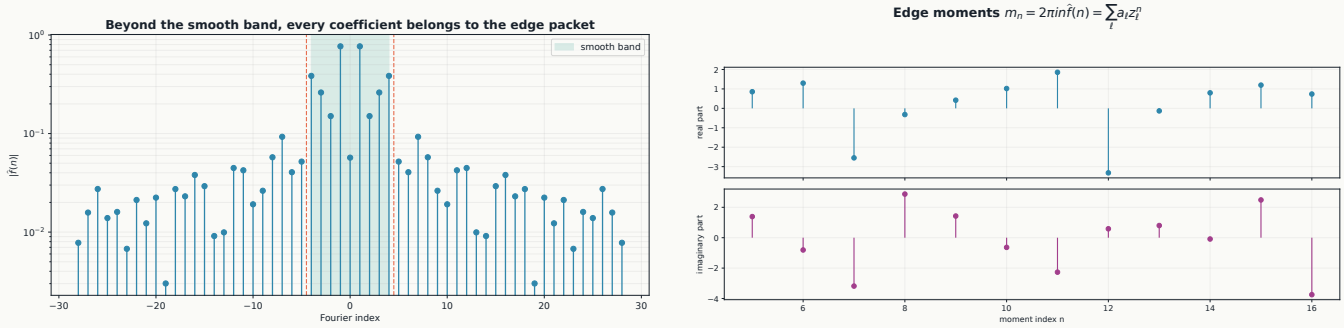
Define the high-frequency edge moments

$$\boxed{m_n = 2\pi i n \widehat{f}(n) \qquad (n > K).}$$

Then

$$\boxed{m_n = \sum_{\ell=1}^{r} a_\ell z_\ell^n.}$$

The background coefficients never enter this formula. The smooth packet has been removed by spectral projection rather than estimated jointly with the edges.



(a) The smooth band and the edge tail

(b) The finite exponential moment sequence

Left. The shaded region contains the unknown smooth background. Every coefficient beyond that region belongs to the edge packet. **Right.** Multiplication by $2\pi i n$ converts the $1/n$ Fourier tail into a finite exponential sum.

10.2 The operator interpretation

Let $P_{>K}$ select Fourier modes with $n > K$. Let D denote distributional differentiation. The complete edge-analysis map is

$$f \longmapsto P_{>K} D f.$$

On the declared class,

$$P_{>K} D f = P_{>K} D j.$$

This is a projector-resolved operation. The smooth component is killed because it belongs to a known invariant spectral packet. The remaining object is not the edge function itself. It is its finite innovation measure.

11 The Hankel packet and its rank

11.1 Hankel construction

Fix

$$n_0 = K + 1.$$

For a positive integer q, form the $q \times q$ Hankel matrix

$$H_q = \left(m_{n_0+p+q'}\right)_{p,q'=0}^{q-1}.$$

When the true number of active jumps is r, define the rectangular Vandermonde matrix

$$V_q = \begin{pmatrix} 1 & \cdots & 1 \\ z_1 & \cdots & z_r \\ \vdots & & \vdots \\ z_1^{q-1} & \cdots & z_r^{q-1} \end{pmatrix}.$$

Then

$$\boxed{H_q = V_q \operatorname{diag}(a_1 z_1^{n_0}, \dots, a_r z_r^{n_0}) V_q^{\mathsf{T}}.}$$

Proof. The (p, q') entry of the right side is

$$\sum_{\ell=1}^{r} z_\ell^p (a_\ell z_\ell^{n_0}) z_\ell^{q'} = \sum_{\ell=1}^{r} a_\ell z_\ell^{n_0+p+q'} = m_{n_0+p+q'}.$$

□

The Hankel packet factors through a Vandermonde matrix and the unknown jumps

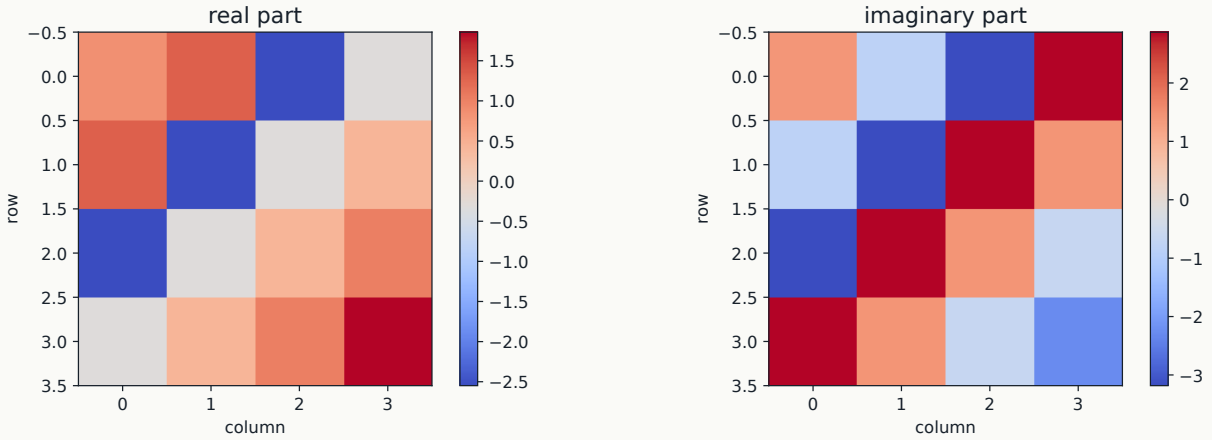


Figure 11.1: A representative Hankel packet.

Reading the figure. The matrix is built from consecutive moments. Its factorization contains the unknown edge locations in the Vandermonde factors and the jump amplitudes in the diagonal factor.

11.2 Unknown number of edges

Theorem 11.1 (Hankel rank equals the number of active jumps)**.** *Assume the jump locations are distinct and the jump amplitudes are nonzero. If $q \geq r$, then*

$$\boxed{\operatorname{rank} H_q = r.}$$

Proof. The matrix V_q has full column rank because the nodes z_ℓ are distinct. The diagonal matrix is invertible because every a_ℓ and z_ℓ is nonzero. The transpose V_q^{T} has full row rank. Since V_q is injective on $\mathbb{C}^r$, multiplication on the left cannot reduce the rank of the full-row-rank product of the remaining two factors. Therefore the product has rank r. □

Corollary 11.2 (Recovery of an unknown edge count)**.** *Suppose only an upper bound L is known. Construct H_L from $2L-1$ consecutive moments. In exact arithmetic,*

$$r = \operatorname{rank} H_L.$$

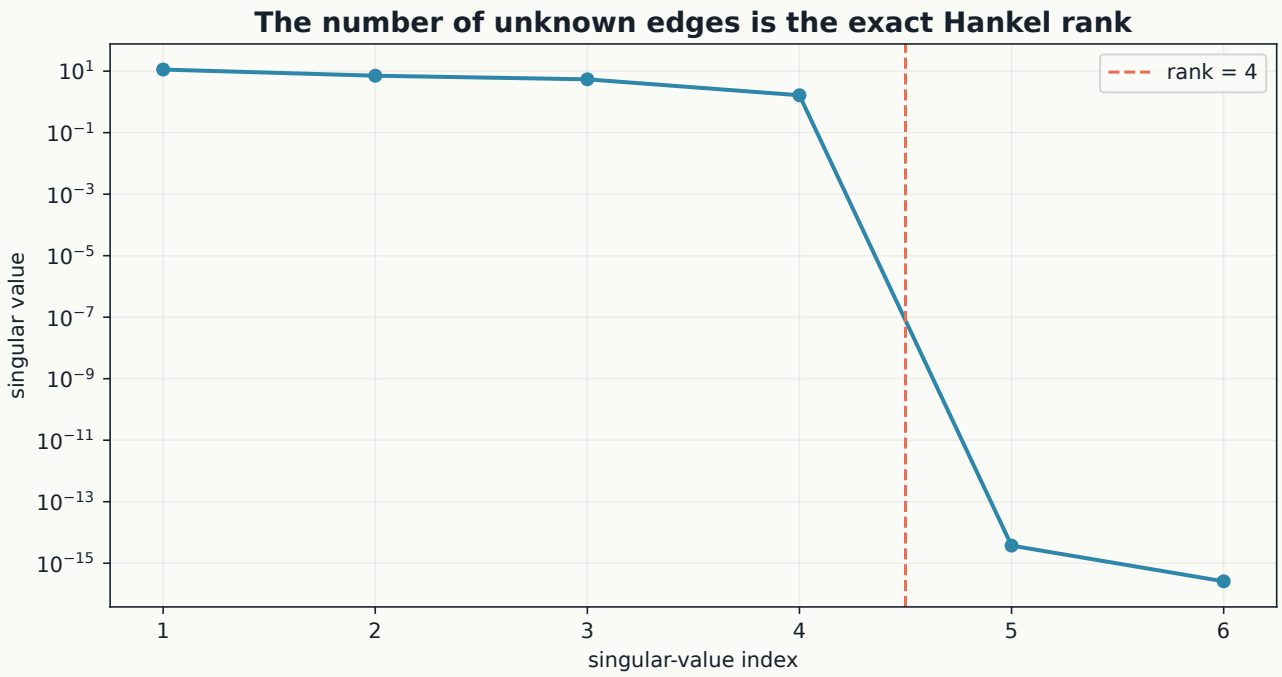


Figure 11.2: The unknown number of edges appears as a rank break.

Reading the figure. The representative example was supplied with an upper bound of six. The Hankel matrix has exactly four significant singular values because the signal has four active jumps.

12 The annihilating polynomial

12.1 Construction

Define the monic polynomial

$$\boxed{Q(z) = \prod_{\ell=1}^{r} (z - z_\ell) = q_0 + q_1 z + \cdots + q_{r-1} z^{r-1} + z^r.}$$

Because $Q(z_\ell) = 0$,

$$\begin{aligned} \sum_{k=0}^{r} q_k m_{n+k} &= \sum_{k=0}^{r} q_k \sum_{\ell=1}^{r} a_\ell z_\ell^{n+k} \\ &= \sum_{\ell=1}^{r} a_\ell z_\ell^{n} \sum_{k=0}^{r} q_k z_\ell^{k} \\ &= \sum_{\ell=1}^{r} a_\ell z_\ell^{n} Q(z_\ell) \\ &= 0. \end{aligned}$$

Thus the edge moments satisfy a linear recurrence of order r.

12.2 Finite linear system

Using $n = n_0, \dots, n_0 + r - 1$, the recurrence gives

$$\begin{pmatrix} m_{n_0} & m_{n_0+1} & \cdots & m_{n_0+r-1} \\ m_{n_0+1} & m_{n_0+2} & \cdots & m_{n_0+r} \\ \vdots & \vdots & & \vdots \\ m_{n_0+r-1} & m_{n_0+r} & \cdots & m_{n_0+2r-2} \end{pmatrix} \begin{pmatrix} q_0 \\ q_1 \\ \vdots \\ q_{r-1} \end{pmatrix} = - \begin{pmatrix} m_{n_0+r} \\ m_{n_0+r+1} \\ \vdots \\ m_{n_0+2r-1} \end{pmatrix}.$$

The coefficient matrix is H_r, which is invertible by the rank theorem. Hence the annihilator is uniquely determined.

Theorem 12.1 (Exact location recovery)**.** *Under the hypotheses above, $2r$ consecutive high-frequency moments determine Q uniquely. The roots of Q are exactly*

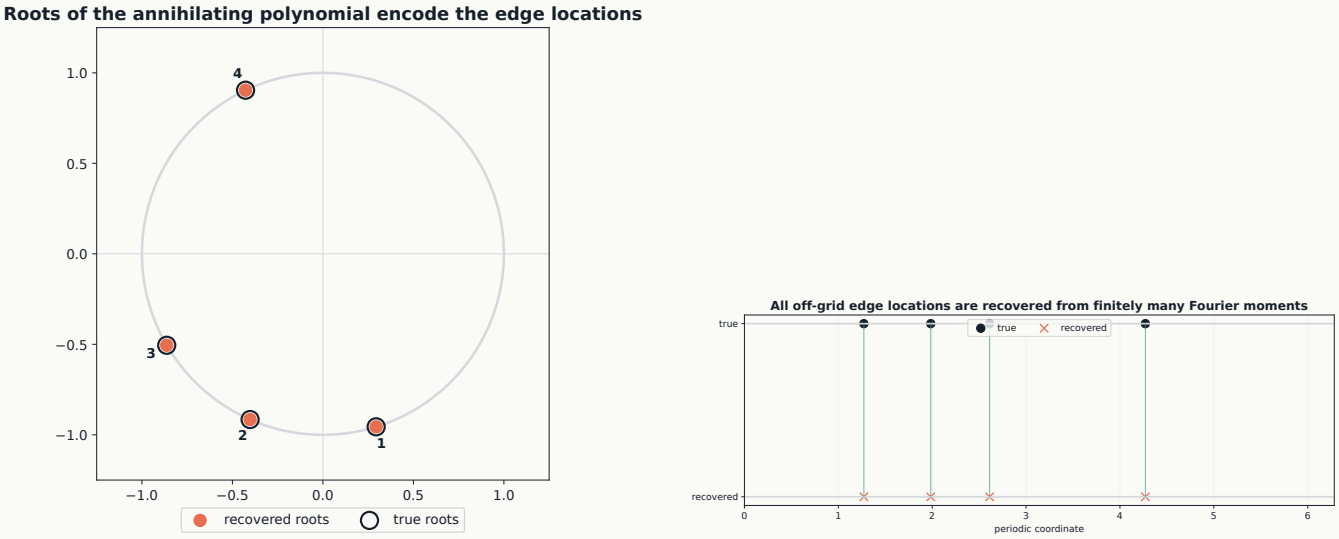
$$z_\ell = \mathrm{e}^{-\mathrm{i}\tau_\ell}.$$

Therefore

$$\boxed{\tau_\ell = -\arg z_\ell \pmod{2\pi}}$$

recovers every unknown edge location.

Proof. The Hankel system has one solution because H_r is invertible. The true monic polynomial satisfies the system, so the computed polynomial equals the true polynomial. Its roots are the distinct values z_ℓ. □



(a) Roots on the unit circle (b) True and recovered edge positions

Left. The roots of the computed polynomial lie on the same points as the true edge nodes. **Right.** The off-grid coordinates agree to roughly 10^{-14} in the representative double-precision example.

13 Recovering jump amplitudes and the edge packet

13.1 Vandermonde solve

Once z_ℓ are known, the first r moments give

$$\begin{pmatrix} m_{n_0} \\ m_{n_0+1} \\ \vdots \\ m_{n_0+r-1} \end{pmatrix} = \begin{pmatrix} z_1^{n_0} & \cdots & z_r^{n_0} \\ z_1^{n_0+1} & \cdots & z_r^{n_0+1} \\ \vdots & & \vdots \\ z_1^{n_0+r-1} & \cdots & z_r^{n_0+r-1} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_r \end{pmatrix}.$$

This Vandermonde matrix is invertible because the nodes are distinct.

Theorem 13.1 (Exact jump-amplitude recovery)**.** *The recovered locations and any r consecutive moments determine the jump amplitudes uniquely.*

Proof. Factor the matrix as

$$\begin{pmatrix} 1 & \cdots & 1 \\ z_1 & \cdots & z_r \\ \vdots & & \vdots \\ z_1^{r-1} & \cdots & z_r^{r-1} \end{pmatrix} \operatorname{diag}(z_1^{n_0}, \ldots, z_r^{n_0}).$$

Both factors are invertible. □

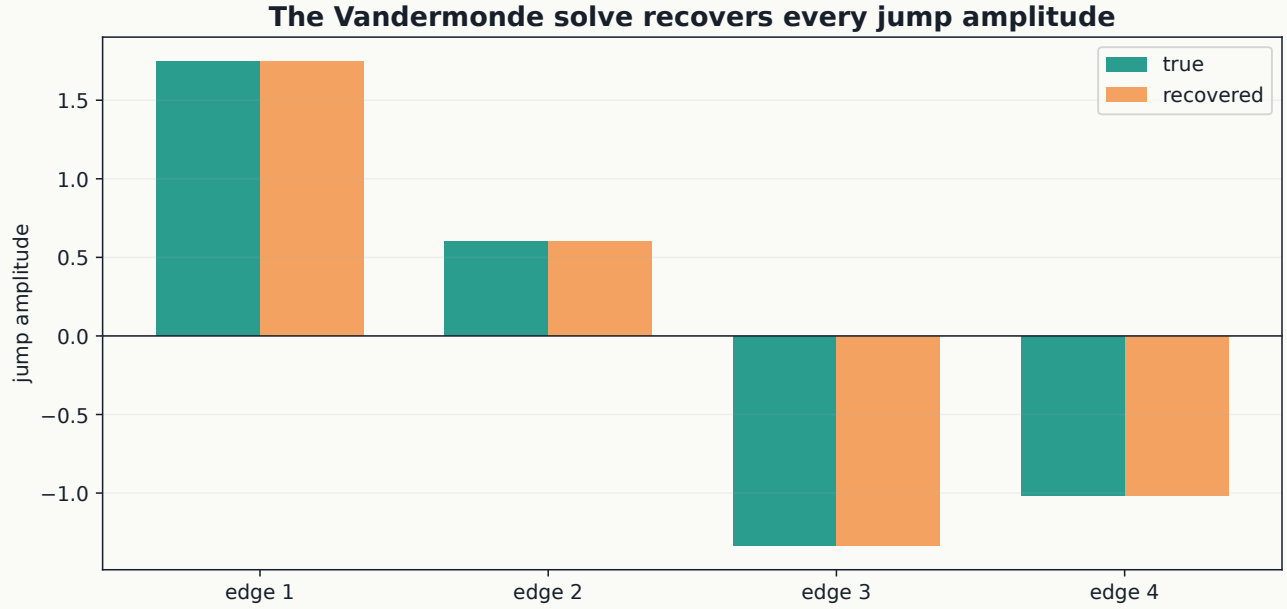


Figure 13.1: Exact jump-amplitude recovery.

Reading the figure. The true and recovered bars overlap at plotting precision. The code uses all available moments in a least-squares solve, which agrees with the exact square system in the noise-free case.

13.2 Periodic integration

Sort the locations so that

$$0 \le \tau_1 < \cdots < \tau_r < 2\pi.$$

Let

$$\ell_k = \tau_{k+1} - \tau_k, \qquad \tau_{r+1} = \tau_1 + 2\pi,$$

and define cumulative levels

$$c_k = \sum_{j=1}^{k} a_j.$$

A piecewise-constant function with the correct derivative has value $b + c_k$ on $[\tau_k, \tau_{k+1})$. The zero-mean normalization determines

$$\boxed{b = -\frac{1}{2\pi} \sum_{k=1}^{r} \ell_k c_k.}$$

This reconstructs the complete edge packet j.

Remark 13.2. The derivative determines a periodic piecewise-constant function up to a constant. That constant is not an edge parameter. Assigning it to the smooth packet gives a canonical direct sum.

14 Recovering the smooth packet

For $|n| \le K$, compute

$$\boxed{\widehat{s}(n) = \widehat{f}(n) - \widehat{j}(n).}$$

For $|n| > K$, set $\widehat{s}(n) = 0$. Then

$$s(x) = \sum_{|n| \le K} \widehat{s}(n) \mathrm{e}^{\mathrm{i}nx}.$$

Every background coefficient is therefore recovered by direct subtraction after the edge packet is known.

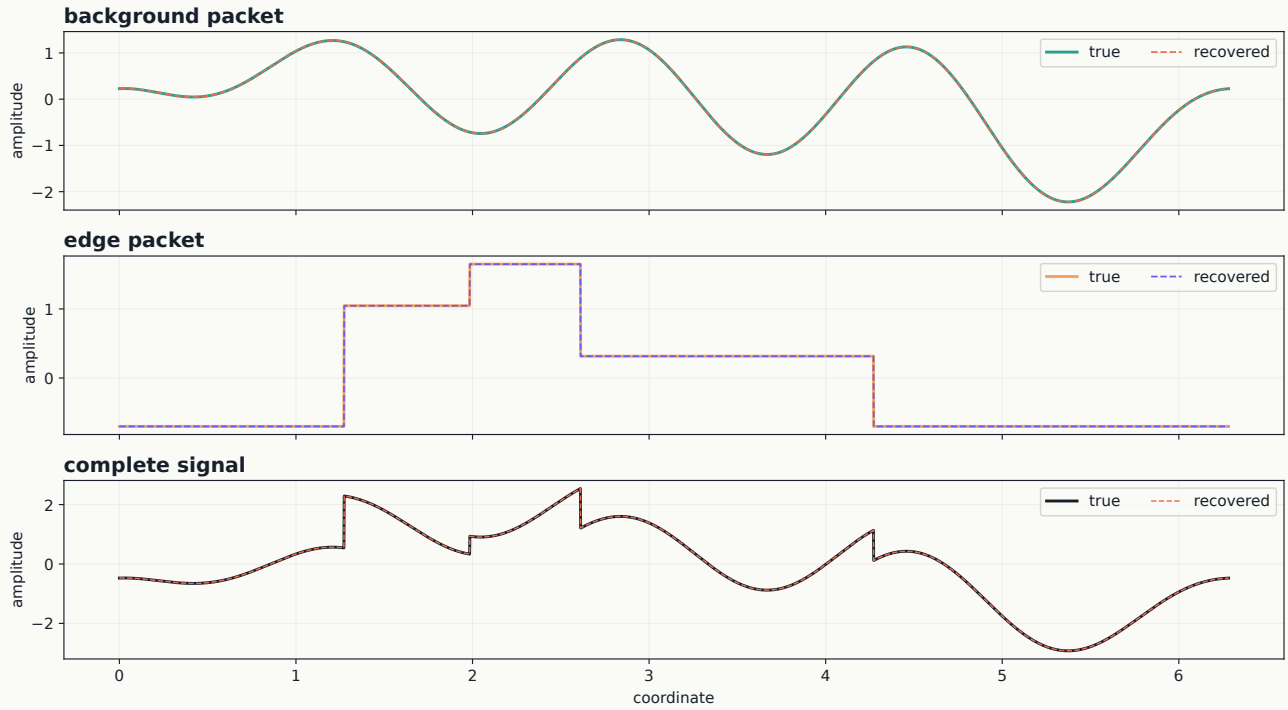


Figure 14.1: Recovered smooth, edge, and complete packets.

Reading the figure. The top row compares the true and recovered background. The middle row compares the edge packet. The bottom row shows complete synthesis. The dashed curves lie on the originals.

15 The universal off-grid recovery theorem

Theorem 15.1 (Universal unknown-edge recovery in $\mathcal{U}_{K,L}$)**.** *Fix upper bounds K and L. Let $f \in \mathcal{U}_{K,L}$ have $r \le L$ distinct active jumps. Assume the jump amplitudes are nonzero. Suppose the Fourier coefficients*

$$\widehat{f}(n)$$

are known for

$$-K \le n \le K$$

and for the $2L$ consecutive positive indices

$$K+1, K+2, \dots, K+2L.$$

Then the following objects are determined uniquely and can be constructed by finite linear algebra.

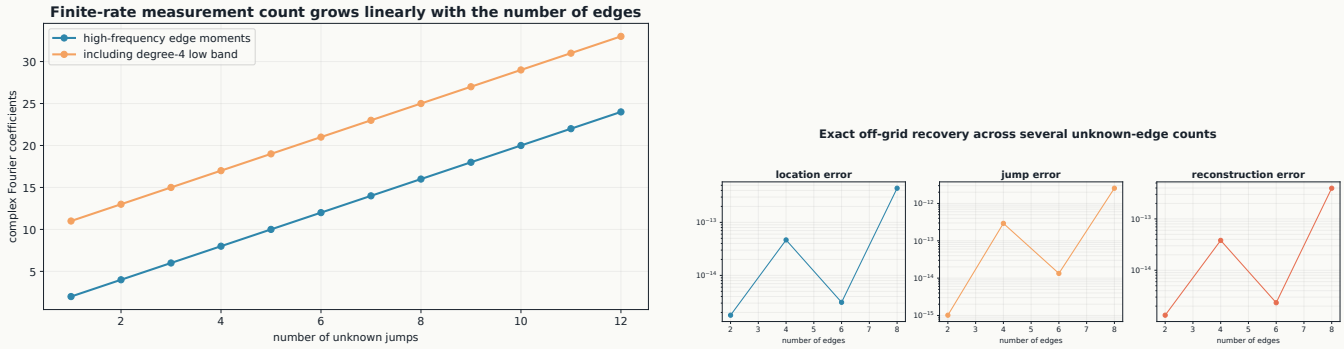
1. *The actual number of jumps r.*
2. *Every off-grid jump location τ_ℓ.*
3. *Every jump amplitude a_ℓ.*
4. *The canonical zero-mean edge packet j.*
5. *Every background Fourier coefficient $\widehat{s}(n)$.*
6. *The complete signal $f = s + j$.*

Proof. Form the high-frequency moments

$$m_n = 2\pi \mathrm{i} n \widehat{f}(n).$$

Since $n > K$, these equal the edge moments. Build the $L \times L$ Hankel matrix. Its rank is the true number r. Use the leading $r \times r$ Hankel system to recover the monic annihilating polynomial. Its distinct roots yield the locations. The Vandermonde system yields the jump amplitudes. Periodic integration with zero mean yields j. Subtraction from the low Fourier modes yields s. Their sum is f. □

Corollary 15.2 (Finite-rate measurement count)**.** *The high-frequency edge information requires $2L$ complex Fourier coefficients. The smooth packet requires $2K + 1$ low-frequency coefficients. Thus the declared class is reconstructed from a number of coefficients that grows linearly in the number of edge and background degrees of freedom.*



(a) Measurement count (b) Four exact random examples

Left. The number of required Fourier coefficients grows linearly with the edge bound. **Right.** Signals with two, four, six, and eight off-grid jumps were reconstructed at floating-point precision.

15.1 Representative numerical values

For the four-edge case used throughout the figures,

$$\max_\ell |\widehat{\tau}_\ell - \tau_\ell|_{\mathbb{T}} = 4.66 \times 10^{-14},$$

$$\max_\ell |\widehat{a}_\ell - a_\ell| = 2.93 \times 10^{-13},$$

and

$$\left\| \widehat{f} - f \right\|_{L^\infty(8192 \text{ points})} = 3.82 \times 10^{-14}.$$

Across all four exact random cases, the largest location error was 4.41×10^{-13}, the largest jump error was 2.56×10^{-12}, and the largest reconstruction error was 3.96×10^{-13}.

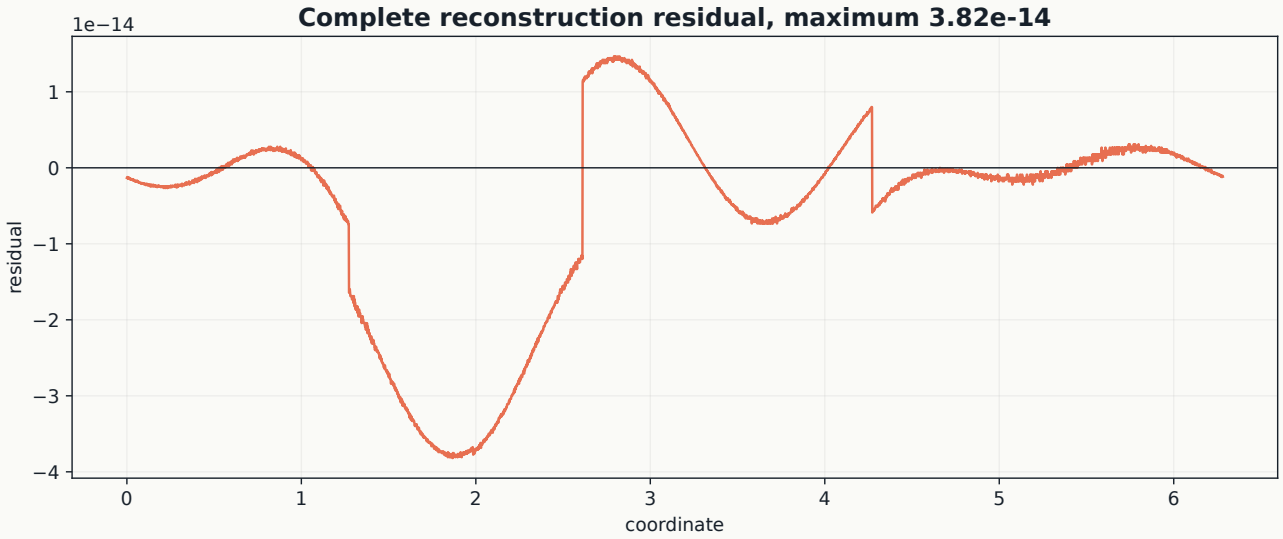


Figure 15.2: The final reconstruction residual.

Reading the figure. The residual is numerical roundoff. Its shape reflects floating-point root and Vandermonde errors, not a missing edge packet.

Part IV: Gibbs-free synthesis after unknown-edge recovery

16 Why exact edge recovery changes the reconstruction problem

16.1 The raw Fourier route

A discontinuity has Fourier coefficients that decay only as $1/|n|$. A raw partial sum therefore asks a globally smooth trigonometric polynomial to reproduce a jump. The Dirichlet kernel supplies the familiar oscillatory boundary layer. Increasing the cutoff narrows that layer, but it does not remove the limiting relative overshoot.

For the recovered model

$$f = s + j,$$

the raw order-N Fourier reconstruction is

$$S_N f = S_N s + S_N j.$$

When $N \geq K$, the smooth term is already exact:

$$S_N s = s.$$

The entire remaining error is therefore

$$S_N f - f = S_N j - j.$$

The Gibbs phenomenon is not spread throughout the model. It is carried by the edge packet.

16.2 The resolved synthesis route

Once the edge parameters are recovered, define

$$\widehat{j} = \mathrm{Integrate}_0\left(\sum_{\ell=1}^{r} \widehat{a}_\ell \delta_{\widehat{\tau}_\ell}\right),$$

where $\mathrm{Integrate}_0$ denotes periodic integration with zero mean. Define the recovered smooth packet by

$$\widehat{s}(x) = \sum_{|n|\le K} \left(\widehat{f}(n) - \widehat{\widehat{j}}(n)\right) \mathrm{e}^{\mathrm{i}nx}.$$

The final synthesis is

$$\boxed{\widehat{f}_{\mathrm{resolved}} = \widehat{s} + \widehat{j}.}$$

The edge is never sent through a finite global Fourier truncation. It is restored from its recovered jump data.

Theorem 16.1 (Gibbs-free exact synthesis)**.** *Under the hypotheses of the universal off-grid recovery theorem, the resolved reconstruction satisfies*

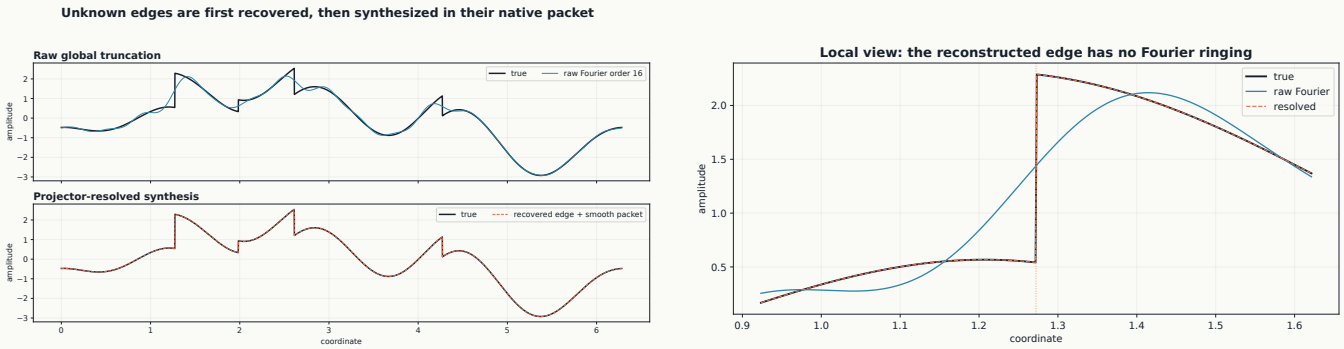
$$\widehat{f}_{\mathrm{resolved}} = f.$$

In particular, the final signal has no Gibbs overshoot or undershoot.

Proof. The recovery theorem gives the exact locations and amplitudes, hence $\widehat{j} = j$. It also gives every low-frequency background coefficient, hence $\widehat{s} = s$. Therefore

$$\widehat{f}_{\mathrm{resolved}} = \widehat{s} + \widehat{j} = s + j = f.$$

The conclusion concerns the synthesized object. A plot of the truncated Fourier series may still show Gibbs ringing, but that approximation is no longer the reconstruction used by the model. □



(a) Raw and resolved synthesis (b) Boundary detail

Left. The raw Fourier reconstruction rings because the discontinuity remains inside the global spectral packet. The resolved reconstruction restores the edge directly. **Right.** The magnified boundary shows that the resolved curve follows the declared one-sided values while the raw partial sum overshoots and oscillates.

16.3 Error decomposition when the edge is only approximate

The exact theorem has a useful perturbative form. Let

$$\widehat{f} = \widehat{s} + \widehat{j}.$$

Then for any norm in which the two errors are defined,

$$\boxed{\left\|f-\widehat{f}\right\| \le \|s-\widehat{s}\| + \left\|j-\widehat{j}\right\|.}$$

For interval or finite-jump packets, the second term can be split into location and amplitude errors. If

$$j = a\mathbf{1}_{\Omega}, \qquad \widehat{j} = \widehat{a}\mathbf{1}_{\widehat{\Omega}},$$

then for $1 \le p < \infty$,

$$\boxed{\left\|j-\widehat{j}\right\|_{L^p} \le |a-\widehat{a}|\,\mu(\Omega)^{1/p} + |\widehat{a}|\,\mu(\Omega\triangle\widehat{\Omega})^{1/p}.}$$

This is the relevant distinction. A wrong edge packet does not mysteriously become exact. Its support and amplitude errors remain visible in the final residual.

17 The closed recovery algorithm

Algorithm 17.1 (Unknown-edge recovery and resolved synthesis)**.** *Given a bandlimit bound K, an edge-count bound L, and Fourier data at the required indices, perform the following steps.*

1. *Form the high-frequency derivative moments*
$$m_n = 2\pi \mathrm{i} n \widehat{f}(n), \qquad n = K+1, \dots, K+2L.$$
2. *Build the $L \times L$ Hankel matrix from consecutive moments.*
3. *Determine the active jump count from its exact rank, or from a declared singular-value threshold in noisy data.*
4. *Solve the square Hankel recurrence for the monic annihilating polynomial.*
5. *Find its roots and map them to circular locations by*
$$\tau_\ell = -\arg z_\ell \pmod{2\pi}.$$
6. *Solve the Vandermonde system for the jump amplitudes.*
7. *Integrate the recovered derivative packet with zero mean to obtain j.*
8. *Subtract the Fourier coefficients of j from the low modes of f to recover s.*
9. *Synthesize*
$$\widehat{f} = s + j.$$
10. *Report the recurrence residual, amplitude residual, root separation, condition numbers, and final reconstruction residual.*

The algorithm is finite. Its nonlinear step is polynomial root finding. Every other step is a matrix construction, a rank computation, a linear solve, or a direct synthesis formula.

The full sequence is summarized in Figure 1.1. Every arrow in that diagram corresponds to a formula proved in this book.

Part V: Stability, conditioning, and the exact boundary

18 Why exact recovery is not the same as uniform stability

18.1 Fourier noise becomes derivative-moment noise

Suppose the observed high Fourier coefficients are

$$\widehat{f_\eta}(n) = \widehat{f}(n) + \eta_n.$$

The derivative moments become

$$m_n^\eta = 2\pi \mathrm{i} n \widehat{f_\eta}(n) = m_n + 2\pi \mathrm{i} n \eta_n.$$

Thus differentiation amplifies coefficient noise by a factor proportional to $|n|$. High modes remove the bandlimited background, but they are not free of numerical cost.

18.2 Hankel perturbation

Let

$$H = H(m), \qquad H_\eta = H(m+e) = H + E.$$

Weyl's inequality gives

$$|\sigma_k(H_\eta) - \sigma_k(H)| \le \|E\|_2 .$$

If the true Hankel rank is r, then a sufficient rank-identification condition is

$$\boxed{\|E\|_2 < \frac{1}{2}\sigma_r(H).}$$

Under this condition, the first r singular values remain separated from the zero tail by a nonzero gap. The rank threshold is therefore tied to the smallest active singular value, not chosen independently of the signal geometry.

18.3 Root localization by Rouché's theorem

The annihilator has the form

$$Q(z) = \prod_{\ell=1}^{r} (z - z_\ell),$$

with distinct roots on the unit circle. Let

$$\widetilde{Q}(z) = Q(z) + E_Q(z)$$

be the polynomial obtained from perturbed moments.

Choose $0 < \rho < \Delta/2$, where

$$\Delta = \min_{j \neq k} |z_j - z_k|.$$

The disks

$$D_j = \{z : |z - z_j| < \rho\}$$

are disjoint. Define

$$m_j(\rho) = \min_{|z - z_j| = \rho} |Q(z)|.$$

Theorem 18.1 (Deterministic root-disk certificate)**.** *If*

$$\sup_{|z-z_j|=\rho} |E_Q(z)| < m_j(\rho)$$

for every j, then $\widetilde{Q}$ has exactly one root in each disk D_j. Hence the recovered roots can be matched to the true roots with error less than ρ.

Proof. On the boundary of D_j, the perturbation has smaller modulus than Q. Rouché's theorem gives the same number of zeros inside the disk. Since Q has exactly one simple zero there, so does $\widetilde{Q}$. □

If

$$E_Q(z) = \sum_{k=0}^{r-1} \delta q_k z^k,$$

then a computable sufficient condition is

$$\sum_{k=0}^{r-1} |\delta q_k|(1+\rho)^k < \min_j m_j(\rho).$$

The condition becomes more demanding as roots approach each other because the boundary minima shrink.

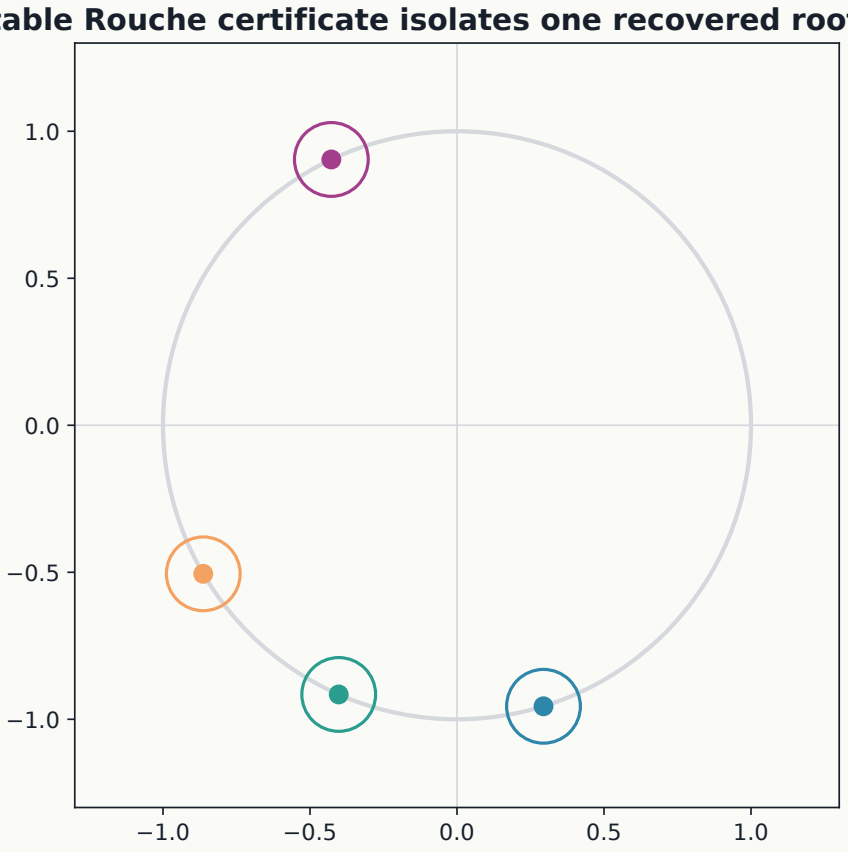


Figure 18.1: Certified root neighborhoods.

Reading the figure. Disjoint disks around simple annihilator roots provide a concrete local stability certificate. The disks cannot remain uniformly large when edge locations collide.

18.4 Amplitude stability

After location recovery, the amplitudes solve

$$Va = m.$$

For a square nonsingular Vandermonde matrix, the first-order perturbation scale is controlled by

$$\kappa(V) = \|V\|_2 \left\|V^{-1}\right\|_2 .$$

More precisely, if V is fixed and only the moments are perturbed, then

$$\|\widetilde{a} - a\|_2 \le \left\|V^{\dagger}\right\|_2 \|\widetilde{m} - m\|_2 .$$

When the locations are also perturbed, an additional matrix perturbation term appears. Both effects worsen as the Vandermonde columns become nearly dependent.

19 Collision is the unavoidable hard boundary

Consider two opposite jumps separated by δ:

$$Dj_\delta = \delta_0 - \delta_\delta.$$

Their moment is

$$m_n(\delta) = 1 - \mathrm{e}^{-\mathrm{i}n\delta}.$$

For fixed n,

$$|m_n(\delta)| = 2\left|\sin\frac{n\delta}{2}\right| \le |n|\delta.$$

Thus every finite block of moments tends to zero with δ, even though the jump pair remains nonzero for every $\delta > 0$. This rules out a uniform inverse Lipschitz constant over all separations.

Theorem 19.1 (No separation-free stability)**.** *Fix finitely many Fourier indices. There is no constant C such that every two-jump periodic edge packet obeys*

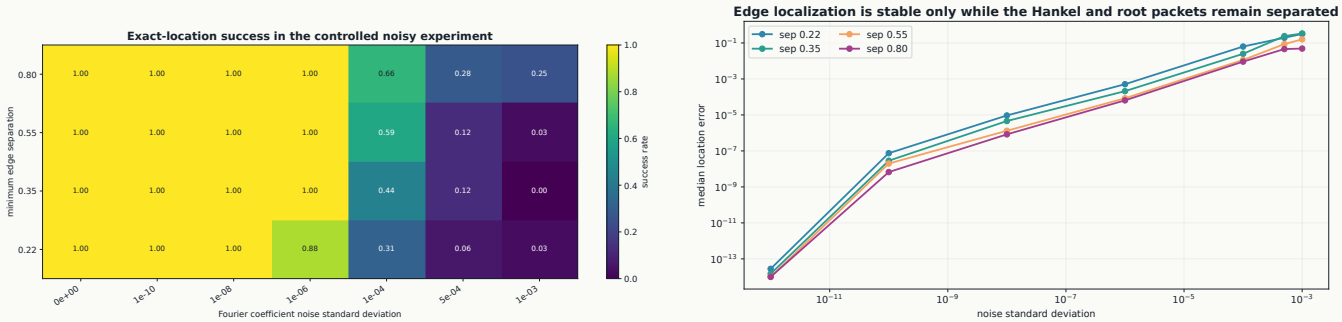
$$\mathrm{dist}(\textit{edge parameters}, 0) \le C\,\|\textit{measured moments}\| .$$

Proof. Use the opposite-jump family above. The parameter vector retains two nonzero unit jumps while the finite moment vector tends to zero as $\delta \to 0$. □

The numerical collision study appears in Figure 3.1b, while the exact moment-collapse witness appears in Figure 3.1a.

20 Finite noise experiments

The implementation studies four minimum-separation regimes and seven Fourier-noise levels. Each table entry is based on repeated deterministic fixed-seed trials. A run is counted as successful when the recovered order is correct and every matched location lies within the declared tolerance.



(a) Recovery frequency

(b) Location error

Left. Larger separation supports a wider stable noise range. The exact zero-noise theorem succeeds in every tested case. **Right.** The median location error rises with the noise level and grows faster when the edge geometry is less separated.

Table 20.1: Selected rows from the noise and separation experiment.

Min. sep.	Noise	Success	Median location	Median RMSE
0.22	10^{-8}	1.000	9.35×10^{-6}	2.00×10^{-6}
0.22	10^{-6}	0.875	5.10×10^{-4}	1.33×10^{-4}
0.22	10^{-4}	0.3125	6.34×10^{-2}	7.49×10^{-2}
0.35	10^{-6}	1.000	2.10×10^{-4}	1.37×10^{-4}
0.55	10^{-6}	1.000	8.45×10^{-5}	2.49×10^{-5}
0.80	10^{-6}	1.000	6.45×10^{-5}	1.53×10^{-5}
0.80	10^{-4}	0.65625	9.20×10^{-3}	3.38×10^{-2}

The experiment is evidence about the stated implementation and parameter range. It does not replace a probabilistic minimax theorem. The deterministic root-disk and singular-value certificates explain the observed transition.

Part VI: Extensions of the same recovery law

21 Several channels with shared edge locations

Suppose C signals share the same edge locations but have channel-dependent jump amplitudes:

$$m_n^{(c)} = \sum_{\ell=1}^{r} a_{c\ell} z_\ell^n.$$

Every channel is annihilated by the same polynomial

$$Q(z) = \prod_{\ell=1}^{r} (z - z_\ell).$$

Stack the channel Hankel equations into one linear system.

Theorem 21.1 (Shared-edge multichannel recovery)**.** *If the stacked recurrence matrix has column rank r, then the common annihilator is unique. Its roots recover every shared location. Channelwise Vandermonde solves then recover the amplitudes in each channel.*

Proof. Each channel supplies linear equations for the same recurrence coefficients. Full column rank makes the stacked solution unique. The root and amplitude steps are then the same as in the scalar theorem. □

A location may be weak or absent in one channel and still be recoverable if it is visible in the stacked system. This is one reason multichannel data can be more stable than a single trace.

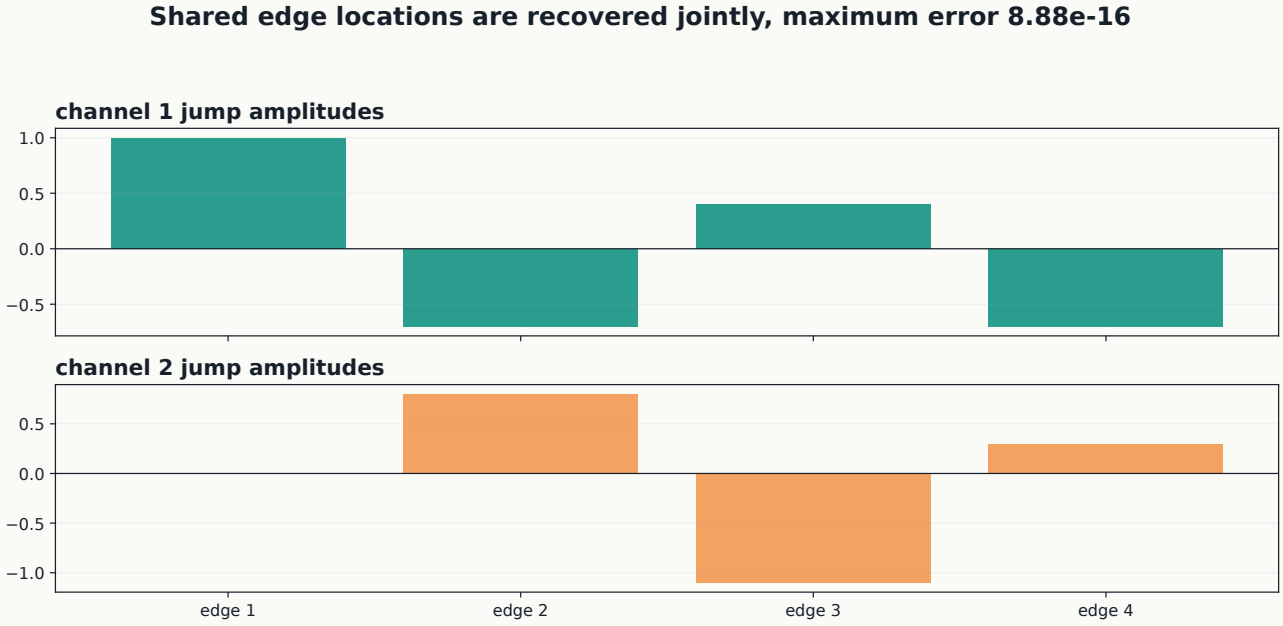


Figure 21.1: Two channels with one shared edge geometry.

Reading the figure. The two signals have different jump amplitudes but the same locations. Stacked Hankel equations recover the common annihilator with a location error of less than 10^{-15} in the displayed example.

22 Piecewise-polynomial packets

Let u be periodic and piecewise polynomial of degree at most d, with breakpoints τ_ℓ. Its $(d+1)$-st distributional derivative is supported at those breakpoints:

$$D^{d+1}u = \sum_{\ell=1}^{r} \sum_{q=0}^{d} b_{\ell q}\, \delta_{\tau_\ell}^{(q)}.$$

Consequently, the Fourier-domain derivative moments have the exponential-polynomial form

$$m_n = \sum_{\ell=1}^{r} p_\ell(n) z_\ell^n,$$

where each p_ℓ has degree at most d.

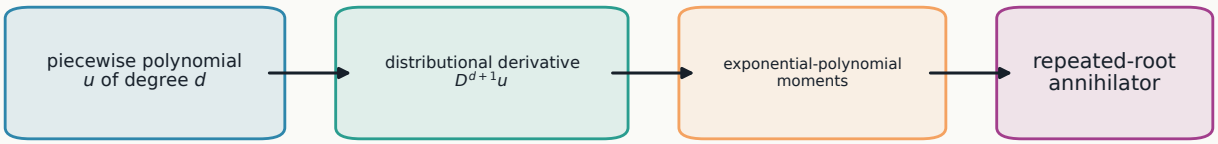
Theorem 22.1 (Confluent annihilation)**.** *Every sequence of the form above is annihilated by*

$$\boxed{Q(E) = \prod_{\ell=1}^{r} (E - z_\ell)^{d+1},}$$

where E is the forward-shift operator on the moment index.

Proof. For one term $n^q z^n$, the operator $E - z$ lowers the polynomial degree by one while retaining the exponential factor. Applying it $q + 1$ times gives zero. The product over all locations annihilates every term. □

The piecewise-constant result is exactly the case $d = 0$. In the polynomial case the roots have multiplicity $d + 1$, so the annihilator order grows to $r(d + 1)$. Recovery then uses confluent Vandermonde systems.



Each edge location becomes a root of multiplicity at most $d + 1$ in the annihilating polynomial.

The piecewise-constant theorem is the case d = 0.

Figure 22.1: From jumps to higher-order innovations.

Reading the figure. Differentiation converts each polynomial segment model into a finite collection of point-supported derivative packets. The annihilator retains the same locations with repeated roots.

23 Two-dimensional finite dictionaries

The finite residualized-secant theorem does not depend on dimension. Let B span a smooth image background and let the columns of D be candidate geometric edge masks. Exact sparse recovery is governed by

$$\operatorname{Ker}((I - P_B)D) \cap \{h : \|h\|_0 \le 2s\} = \{0\}.$$

The implementation uses a small two-dimensional image with a smooth background and one unknown rectangle selected from a complete finite rectangle library. Exhaustive profile least squares recovers the exact rectangle

$$(\text{top}, \text{left}, \text{height}, \text{width}) = (6, 8, 5, 6)$$

and its amplitude, with zero residual to numerical precision.

The complete two-dimensional reconstruction is shown in Figure 7.2.

Boundary 23.1 (Continuous two-dimensional curves). This rectangle calculation does not solve arbitrary continuous two-dimensional edge recovery. A continuous theorem requires a geometric edge model, enough Fourier samples, and a uniqueness condition. One established route models the edge set as the zero set of a trigonometric polynomial and derives a two-dimensional annihilation relation. The present work keeps that larger problem separate from the proved finite dictionary theorem.

Part VII: Failure modes, complexity, and implementation

24 Two ways the exact theorem can be used incorrectly

24.1 Too few moments

An order-r annihilator contains r unknown recurrence coefficients. The square recurrence solve needs at least $2r$ consecutive moments in the direct Prony construction used here. With too few moments, different edge configurations can satisfy the same incomplete data.

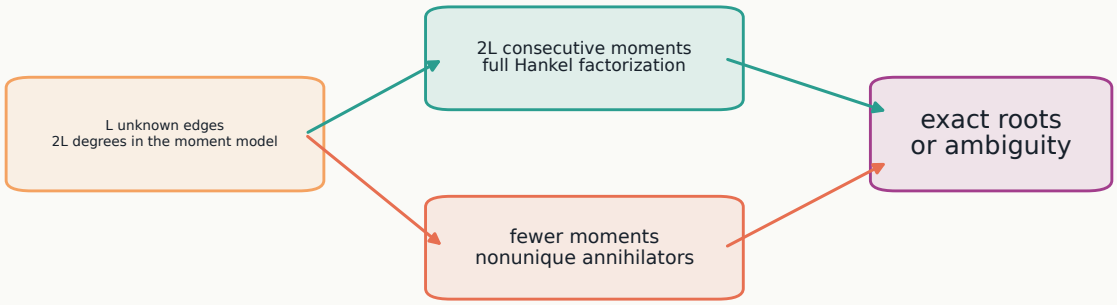


Figure 24.1: The moment-count boundary.

Reading the figure. Below the finite-rate count, the recurrence system is underdetermined. Exact root recovery is then not a theorem, even when a numerical routine returns a polynomial.

24.2 An invalid background bandlimit

The formula

$$m_n = 2\pi i n \widehat{f}(n)$$

is an edge-only moment only when $|n| > K$. Starting inside the true background band contaminates the Hankel packet with smooth Fourier content.

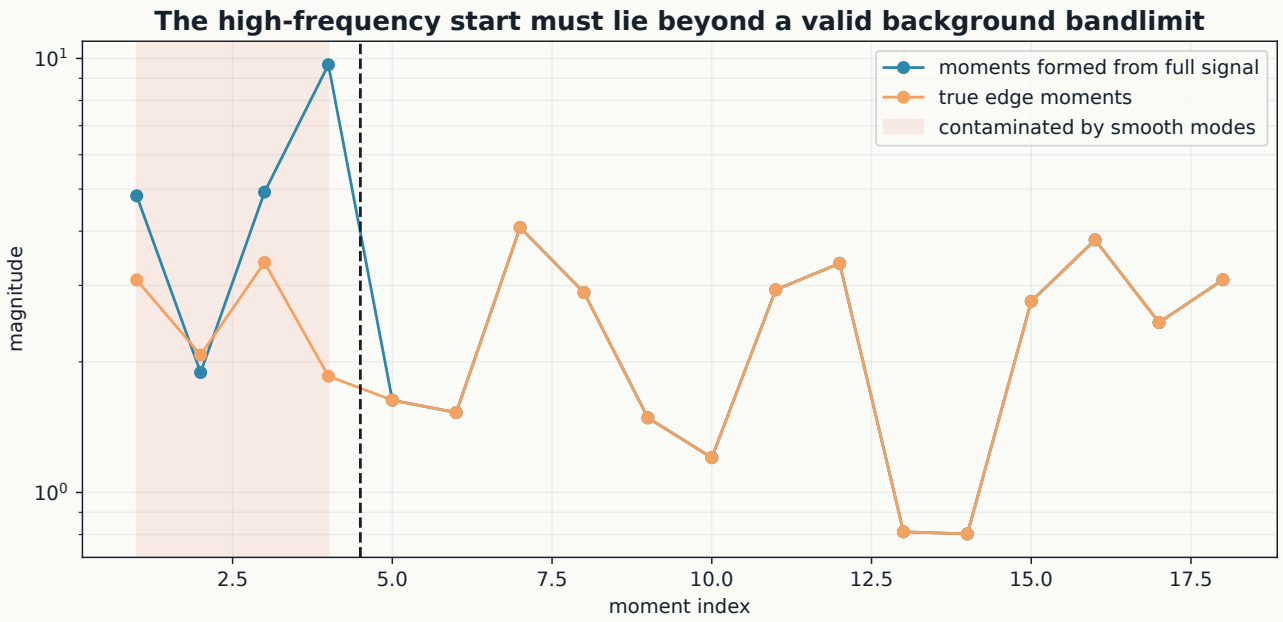


Figure 24.2: The declared bandlimit matters.

Reading the figure. When the high-frequency block begins too early, the background has not vanished. The moment sequence no longer has the assumed finite exponential form.

24.3 Model mismatch

Other sources of mismatch include:

- a background that is not exactly bandlimited
- more jumps than the supplied upper bound
- repeated or nearly colliding jump locations
- a non-piecewise-constant edge packet treated as piecewise constant
- Fourier coefficients computed under a different normalization
- nonperiodic boundary data used without a boundary model

Each mismatch appears in one of the internal certificates: rank tail, recurrence residual, root modulus, amplitude residual, or final reconstruction residual.

25 Computational cost

For known order r, the direct implementation performs:

- an $r \times r$ Hankel solve
- a degree-r polynomial root computation
- a least-squares Vandermonde solve with $2L$ rows
- direct Fourier and edge synthesis

The dense algebraic cost is dominated by cubic operations in r. The measurement count is linear in $K + L$. This distinction matters. The model is sample-efficient because its innovation rate is finite, but a naive dense solver is not claimed to be optimal for large r.

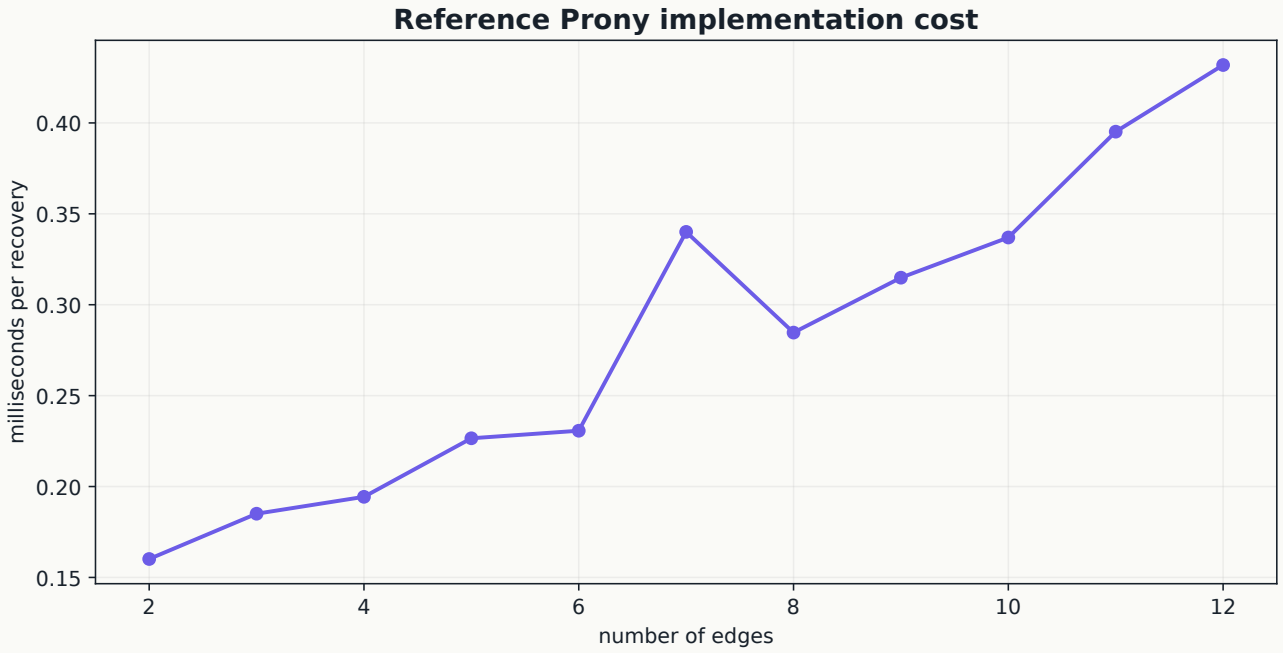


Figure 25.1: Measured direct runtime.

Reading the figure. The measured CPU time remains below one millisecond through order twelve in the supplied environment. This is a small implementation benchmark, not an asymptotic proof. The coefficient count is shown separately in Figure 15.1a.

Table 25.1: Measured direct recovery time.

Edge order	Median seconds
2	0.000164
4	0.000213
6	0.000231
8	0.000275
10	0.000342
12	0.000425

26 A symbolic four-edge calculation

Take the four roots

$$z_1 = 1, \qquad z_2 = \mathrm{i}, \qquad z_3 = -1, \qquad z_4 = -\mathrm{i}.$$

Their annihilator is

$$\boxed{Q(z) = (z-1)(z-\mathrm{i})(z+1)(z+\mathrm{i}) = z^4 - 1.}$$

Choose nonzero jump weights summing to zero. The resulting moment sequence satisfies

$$m_{n+4} - m_n = 0.$$

The exact symbolic Hankel determinant generated by the code is

$$\boxed{6144},$$

so the order-four recurrence system is nonsingular. This calculation checks the root, recurrence, and rank conventions without floating-point arithmetic.

27 Verification ledger

The single implementation performs nineteen named checks. They are not used as substitutes for the proofs. They verify that the displayed formulas and the source code agree.

Table 27.1: Complete computational check ledger.

Check	Result	Recorded detail
Symbolic annihilator	pass	$z^4 - 1$
Distributional Fourier identity	pass	residual 4.97×10^{-16}
Unknown order	pass	true order and recovered order both four
Off-grid locations	pass	maximum error 4.66×10^{-14}
Jump amplitudes	pass	maximum error 2.93×10^{-13}
Annihilating recurrence	pass	residual 6.17×10^{-14}
Vandermonde amplitude solve	pass	residual 3.35×10^{-13}
Full signal reconstruction	pass	error 3.82×10^{-14}
Gibbs-free synthesis	pass	raw Fourier RMSE 2.57×10^{-1}
Four exact random cases	pass	orders two, four, six, and eight
Finite dictionary support	pass	recovered support $\{1, 5\}$
Finite dictionary synthesis	pass	residual 1.21×10^{-15}
Restricted separation	pass	$\gamma_4 = 0.5116895$
Finite dictionary stability	pass	error below deterministic bound
Shared multichannel edges	pass	location error 8.88×10^{-16}
Piecewise-polynomial recurrence	pass	residual 1.59×10^{-14}
Finite-sampling impossibility witness	pass	hidden interval between sample points
Collision instability	pass	moment norm shrinks with separation
Two-dimensional rectangle	pass	exact support and amplitude

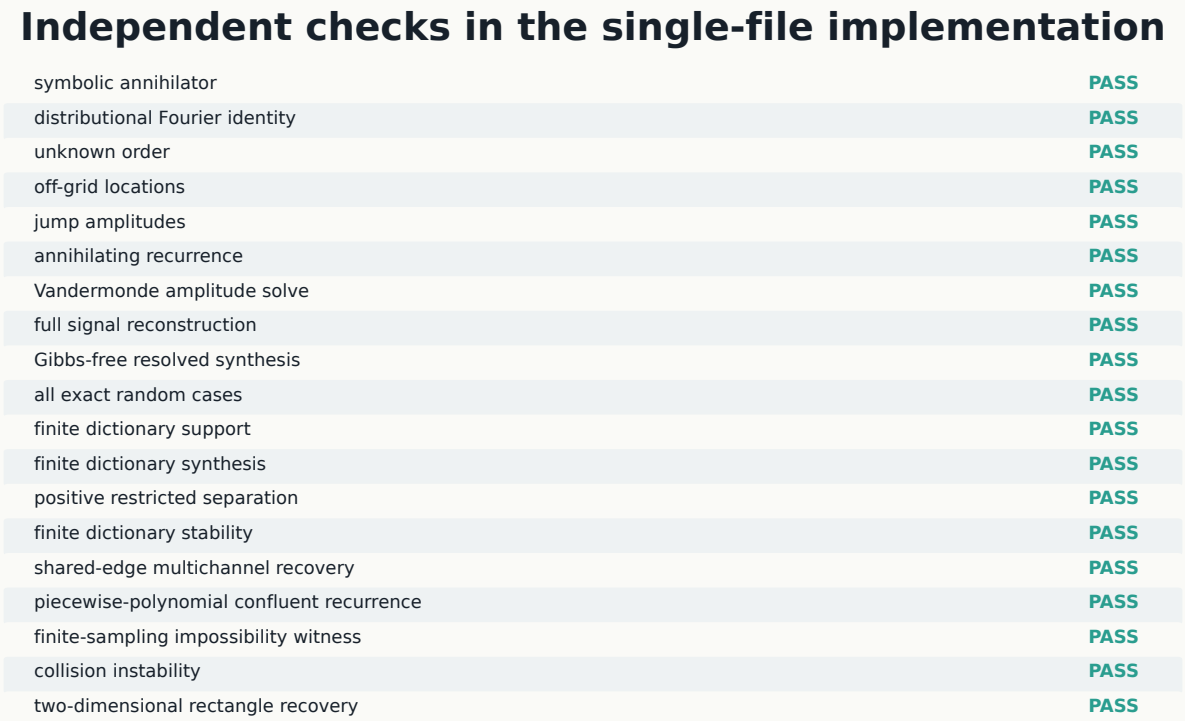


Figure 27.1: The complete finite verification dashboard.

Reading the figure. Every check is tied to a theorem, a boundary example, or a stated implementation contract.

28 How the pieces depend on one another

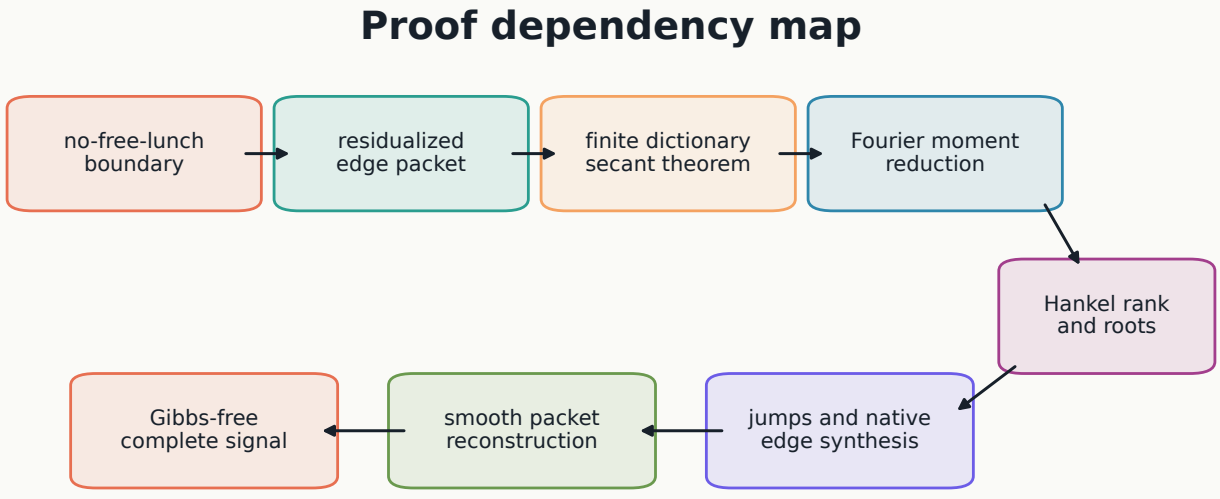


Figure 28.1: Proof dependency map.

Reading the figure. The positive theorem begins only after the impossibility results define the model boundary. The final synthesis depends on successful location, amplitude, edge, and background recovery.

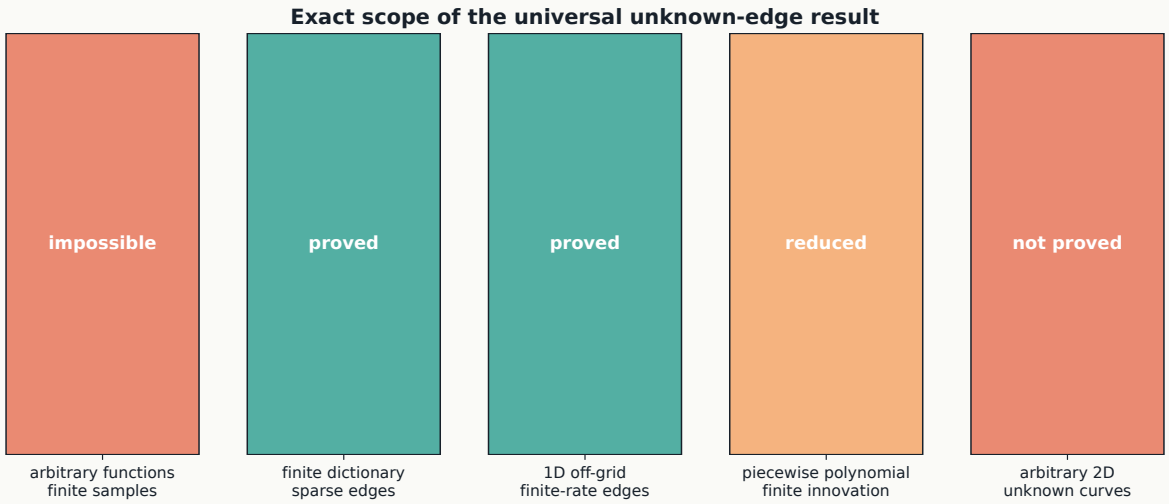


Figure 28.2: What is proved and what remains outside the theorem.

Reading the figure. The green region contains exact finite theorems. The orange region contains local stability conditions and controlled experiments. Arbitrary model-free recovery and arbitrary continuous two-dimensional curves remain outside the proved scope.

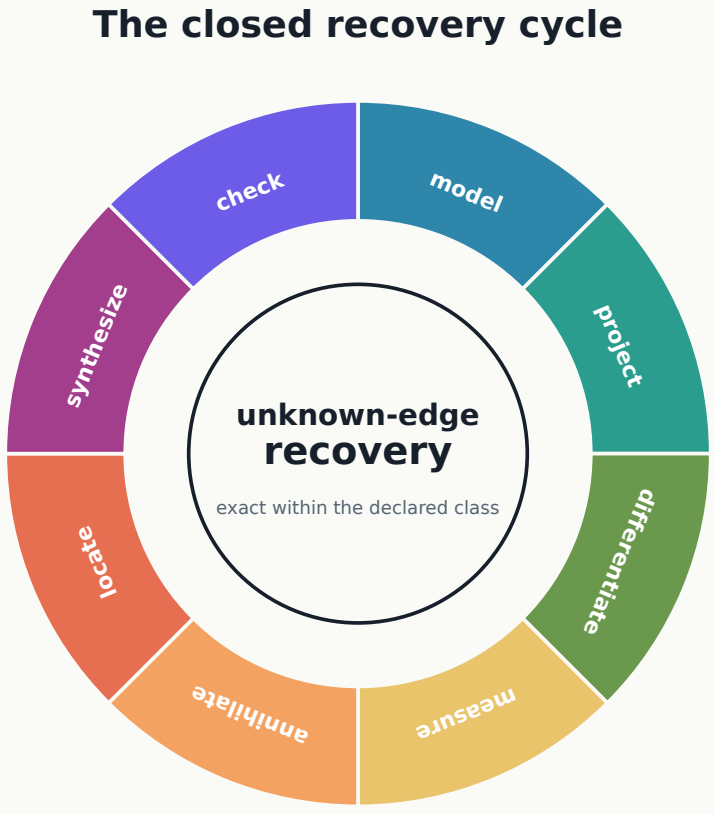


Figure 28.3: The unknown-edge recovery wheel.

Reading the figure. The full loop is measure, project, differentiate, form moments, build the Hankel packet, annihilate, recover geometry, reconstruct packets, and synthesize. The residual returns the process to model diagnosis when an assumption fails.

Part VIII: Relation to established finite-rate recovery

29 What is classical and what is organized here

Annihilating filters, Prony-type recovery, and finite-rate-of-innovation sampling are established ideas. Vetterli, Marziliano, and Blu proved sampling and reconstruction results for signals with a finite rate of innovation, including Dirac streams, nonuniform splines, and piecewise polynomials. Dragotti, Vetterli, and Blu developed compact-support sampling kernels and local reconstruction methods. Candès and Fernandez-Granda proved exact and stable super-resolution results for separated point sources using total-variation minimization. Ongie and Jacob developed off-grid recovery for two-dimensional piecewise-constant images whose edge sets lie on trigonometric curves.

The contribution of this document is narrower and explicit. It places the recovery chain inside the projector-resolved calculus, proves the impossibility boundary before stating a universal claim, joins finite dictionary and off-grid recovery in one language, and carries the recovered edge into a Gibbs-free synthesis certificate. It does not claim invention of Prony's method or finite-rate-of-innovation sampling.

30 The strongest defensible conclusion

There is no model-free solution to universal unknown-edge recovery. There is, however, an exact finite solution for every member of a broad declared class:

$$\boxed{\text{bandlimited periodic background} + \text{finitely many off-grid piecewise-constant jumps.}}$$

Within that class, the unknown edge count, locations, amplitudes, edge packet, background packet, and complete signal are determined by finitely many Fourier coefficients.

The result is universal over the class because the algorithm does not know the individual signal parameters in advance. It uses only the bounds K and L. The theorem is not universal over arbitrary functions, arbitrary measurements, or arbitrarily colliding noisy edges.

The central reconstruction identity is

$$\boxed{\widehat{f} = \widehat{s} + \widehat{j} = f.}$$

This is also the exact answer to the Gibbs problem in the declared model. The global Fourier basis is used where it is exact. The recovered singular packet is synthesized in its own coordinate. Nothing is smoothed across the jump, and no overshoot is introduced into the final reconstruction.

A Detailed exact-recovery table

Table A.1: Exact off-grid recovery across four random cases.

Case	K	True edges	Recovered	Location error	Jump error	Reconstruction error	$\kappa(H)$
1	3	2	2	1.78×10^{-15}	1.02×10^{-15}	1.33×10^{-15}	1.25
2	4	4	4	4.66×10^{-14}	2.93×10^{-13}	3.82×10^{-14}	111.64
3	5	6	6	3.11×10^{-15}	1.34×10^{-14}	2.33×10^{-15}	18.79

Case	K	True edges	Recovered	Location error	Jump error	Reconstruction error	$\kappa(H)$
4	6	8	8	4.41×10^{-13}	2.56×10^{-12}	3.96×10^{-13}	300.16

B Notation and implementation map

Symbol or function	Meaning
$\mathbb{T}$	The periodic coordinate domain $\mathbb{R}/(2\pi\mathbb{Z})$.
$\mathcal{T}_K$	Real trigonometric polynomials of degree at most K.
$\mathcal{U}_{K,L}$	Signals with a degree-K background and at most L active jumps.
Dj	Distributional derivative edge packet.
m_n	Edge moment $2\pi i n \widehat{f}(n)$ above the background band.
H_q	Hankel matrix formed from consecutive moments.
Q	Monic annihilating polynomial.
z_ℓ	Unit-circle root $\mathrm{e}^{-\mathrm{i}\tau_\ell}$.
V	Vandermonde matrix for amplitude recovery.
$P_{\mathcal{S}}$	Orthogonal projector onto a finite background subspace.
$R_{\mathcal{S}}$	Residual projector $I - P_{\mathcal{S}}$.
`recover_unknown_edges`	Hankel rank, annihilator, roots, and amplitudes.
`reconstruct_background`	Low-mode subtraction after edge recovery.
`packet_resolved_reconstruction`	Exact separate synthesis of smooth and edge packets.
`exhaustive_sparse_recovery`	Finite dictionary profile search.
`recover_shared_edges_multichannel`	Stacked recurrence recovery for shared locations.

C Complete single-file implementation

The following source generates the figures, tables, experiments, test ledger, and machine-readable summary used in this book.

```python
from __future__ import annotations

"""Universal unknown-edge recovery in a declared finite-rate packet class.

This single file contains the mathematical model, exact and numerical recovery
algorithms, tests, figures, tables, and machine-readable result records used by
its companion paper.
"""

import argparse
import csv
import hashlib
import json
import math
import time
from dataclasses import dataclass
from itertools import combinations
from pathlib import Path
from typing import Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Circle
import numpy as np
from numpy.typing import NDArray
from scipy.optimize import linear_sum_assignment
from scipy.linalg import subspace_angles
import sympy as sp

Array = NDArray[np.float64]
ComplexArray = NDArray[np.complex128]
PI = math.pi
TAU = 2.0 * math.pi

INK = "#17202A"
MUTED = "#566573"
PAPER = "#FAFAF7"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
RED = "#E76F51"
MAGENTA = "#A23E8C"
PURPLE = "#6C5CE7"
GREEN = "#6A994E"
YELLOW = "#E9C46A"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
PALETTE = [BLUE, TEAL, ORANGE, MAGENTA, PURPLE, GREEN, RED, YELLOW]

plt.rcParams.update(
    {
        "font.family": "DejaVu Sans",
        "font.size": 10.5,
        "axes.titlesize": 14,
        "axes.labelsize": 10.5,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "savefig.facecolor": PAPER,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
    }
)


def wrap_angle(x: NDArray[np.float64] | float) -> NDArray[np.float64] | float:
    return np.mod(x, TAU)


def circular_distance(a: float, b: float) -> float:
    d = abs(a - b) % TAU
    return min(d, TAU - d)


def pairwise_circular_gaps(tau: Array) -> Array:
    t = np.sort(wrap_angle(np.asarray(tau, dtype=float)))
    return np.diff(np.r_[t, t[0] + TAU])


def save_figure(fig: plt.Figure, out_dir: Path, name: str) -> None:
    out_dir.mkdir(parents=True, exist_ok=True)
    fig.savefig(out_dir / f"{name}.pdf", bbox_inches="tight", pad_inches=0.06)
    fig.savefig(out_dir / f"{name}.png", dpi=180, bbox_inches="tight", pad_inches=0.06)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.0, 5.5)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0.0, 1.0)
```

```
    ax.set_ylim(0.0, 1.0)
    ax.axis("off")
    return fig, ax


def draw_box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 11.0,
) -> FancyBboxPatch:
    x, y = xy
    patch = FancyBboxPatch(
        (x, y),
        width,
        height,
        boxstyle="round,pad=0.015,rounding_size=0.02",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.6,
    )
    ax.add_patch(patch)
    label = ax.text(
        x + width / 2,
        y + height / 2,
        text,
        ha="center",
        va="center",
        multialignment="center",
        linespacing=1.12,
        fontsize=fontsize,
        zorder=5,
    )
    fig = ax.figure
    for _ in range(12):
        fig.canvas.draw()
        renderer = fig.canvas.get_renderer()
        box = label.get_window_extent(renderer=renderer)
        (x0, y0), (x1, y1) = ax.transData.transform(((x, y), (x + width, y + height)))
        aw = abs(x1 - x0) * 0.84
        ah = abs(y1 - y0) * 0.80
        if box.width <= aw and box.height <= ah:
            break
        label.set_fontsize(max(7.5, label.get_fontsize() * 0.90))
    return patch


def draw_arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.0),
    connectionstyle: str = "arc3",
) -> FancyArrowPatch:
    arrow = FancyArrowPatch(
        start,
        end,
        arrowstyle="-|>",
        mutation_scale=13,
        linewidth=1.7,
        color=color,
        connectionstyle=connectionstyle,
    )
    ax.add_patch(arrow)
    if text is not None:
```

```
        mx = (start[0] + end[0]) / 2 + text_offset[0]
        my = (start[1] + end[1]) / 2 + text_offset[1]
        ax.text(
            mx,
            my,
            text,
            ha="center",
            va="center",
            fontsize=9.3,
            color=color,
            bbox={"boxstyle": "round,pad=0.12", "facecolor": PAPER, "edgecolor": "none"},
        )
    return arrow


# ---------------------------------------------------------------------------
# Exact signal model
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class EdgeModel:
    """Periodic piecewise-constant packet with canonical zero mean."""

    locations: Array
    jumps: Array

    def __post_init__(self) -> None:
        locations = np.asarray(self.locations, dtype=float)
        jumps = np.asarray(self.jumps, dtype=float)
        if locations.ndim != 1 or jumps.ndim != 1 or locations.size != jumps.size:
            raise ValueError("locations and jumps must be one-dimensional and equally sized")
        if locations.size < 2:
            raise ValueError("a nonconstant periodic packet needs at least two jumps")
        locations = np.mod(locations, TAU)
        order = np.argsort(locations)
        locations = locations[order]
        jumps = jumps[order]
        if np.min(pairwise_circular_gaps(locations)) <= 1e-12:
            raise ValueError("jump locations must be distinct")
        if abs(float(np.sum(jumps))) > 1e-9:
            raise ValueError("periodic jump amplitudes must sum to zero")
        if np.min(np.abs(jumps)) <= 1e-12:
            raise ValueError("zero jump amplitudes must be removed from the model")
        object.__setattr__(self, "locations", locations)
        object.__setattr__(self, "jumps", jumps)

    @property
    def order(self) -> int:
        return int(self.locations.size)

    @property
    def min_separation(self) -> float:
        return float(np.min(pairwise_circular_gaps(self.locations)))

    def fourier(self, n: int) -> complex:
        if n == 0:
            return 0.0 + 0.0j
        return complex(np.sum(self.jumps * np.exp(-1j * n * self.locations)) / (2j * PI * n))

    def moments(self, indices: Sequence[int]) -> ComplexArray:
        n = np.asarray(indices, dtype=int)
        return np.asarray(
            [np.sum(self.jumps * np.exp(-1j * int(k) * self.locations)) for k in n],
            dtype=np.complex128,
        )

    def values(self, x: Array) -> Array:
        xw = np.mod(np.asarray(x, dtype=float), TAU)
        tau = self.locations
        a = self.jumps
        lengths = np.diff(np.r_[tau, tau[0] + TAU])
        cumulative = np.cumsum(a)
```

```
        base = -float(np.sum(lengths * cumulative) / TAU)
        out = np.full_like(xw, base, dtype=float)
        for location, jump in zip(tau, a, strict=True):
            out += jump * (xw >= location)
        # Correct the branch cut. Jumps after tau[0] were applied, but values below
        # tau[0] must also include the jumps from the previous cycle.
        out[xw < tau[0]] += float(np.sum(a))
        return out

    def segment_values(self) -> tuple[Array, Array]:
        tau = self.locations
        lengths = np.diff(np.r_[tau, tau[0] + TAU])
        cumulative = np.cumsum(self.jumps)
        base = -float(np.sum(lengths * cumulative) / TAU)
        return lengths, base + cumulative


@dataclass(frozen=True)
class BandlimitedBackground:
    degree: int
    coefficients: dict[int, complex]

    def __post_init__(self) -> None:
        if self.degree < 0:
            raise ValueError("degree must be nonnegative")
        coeffs = dict(self.coefficients)
        for n in coeffs:
            if abs(n) > self.degree:
                raise ValueError("coefficient outside declared bandlimit")
        for n in range(1, self.degree + 1):
            if abs(coeffs.get(-n, 0.0) - np.conj(coeffs.get(n, 0.0))) > 1e-10:
                raise ValueError("coefficients must define a real signal")
        if abs(complex(coeffs.get(0, 0.0)).imag) > 1e-10:
            raise ValueError("zero-frequency coefficient must be real")
        object.__setattr__(self, "coefficients", coeffs)

    def fourier(self, n: int) -> complex:
        return complex(self.coefficients.get(int(n), 0.0 + 0.0j))

    def values(self, x: Array) -> Array:
        x = np.asarray(x, dtype=float)
        out = np.zeros_like(x, dtype=np.complex128)
        for n, value in self.coefficients.items():
            out += value * np.exp(1j * n * x)
        return np.real_if_close(out, tol=1000).astype(float)


@dataclass(frozen=True)
class UniversalSignal:
    background: BandlimitedBackground
    edge: EdgeModel

    def fourier(self, n: int) -> complex:
        return self.background.fourier(n) + self.edge.fourier(n)

    def values(self, x: Array) -> Array:
        return self.background.values(x) + self.edge.values(x)


@dataclass
class RecoveryResult:
    order: int
    locations: Array
    jumps: Array
    annihilator_ascending: ComplexArray
    hankel_singular_values: Array
    moment_indices: NDArray[np.int64]
    measured_moments: ComplexArray
    recurrence_residual: float
    amplitude_residual: float
    min_root_separation: float
```

```
    hankel_condition: float
    vandermonde_condition: float

    def edge_model(self) -> EdgeModel:
        jumps = np.asarray(np.real(self.jumps), dtype=float)
        jumps -= float(np.mean(jumps))
        # The correction above is at roundoff scale when the periodic condition
        # is satisfied. It keeps the constructor strict in noisy examples.
        return EdgeModel(self.locations, jumps)


def generate_background(degree: int, rng: np.random.Generator, scale: float = 0.35) -> BandlimitedBackground:
    coeffs: dict[int, complex] = {0: float(rng.normal(scale=scale))}
    for n in range(1, degree + 1):
        z = scale * (rng.normal() + 1j * rng.normal()) / math.sqrt(2.0)
        coeffs[n] = z
        coeffs[-n] = np.conj(z)
    return BandlimitedBackground(degree, coeffs)


def generate_edge_model(
    order: int,
    rng: np.random.Generator,
    min_separation: float = 0.28,
    amplitude_scale: float = 1.0,
) -> EdgeModel:
    if order < 2:
        raise ValueError("order must be at least two")
    for _ in range(100000):
        tau = np.sort(rng.uniform(0.0, TAU, order))
        if np.min(pairwise_circular_gaps(tau)) >= min_separation:
            break
    else:
        raise RuntimeError("could not generate separated jump locations")
    jumps = rng.normal(scale=amplitude_scale, size=order)
    jumps -= float(np.mean(jumps))
    while np.min(np.abs(jumps)) < 0.08 * amplitude_scale:
        jumps = rng.normal(scale=amplitude_scale, size=order)
        jumps -= float(np.mean(jumps))
    return EdgeModel(tau, jumps)


def fourier_measurements(signal: UniversalSignal, indices: Sequence[int]) -> dict[int, complex]:
    return {int(n): signal.fourier(int(n)) for n in indices}


def edge_moments_from_fourier(
    measurements: dict[int, complex],
    indices: Sequence[int],
) -> ComplexArray:
    out = []
    for n in indices:
        if n == 0:
            raise ValueError("edge moments use nonzero Fourier indices")
        out.append(2j * PI * n * measurements[int(n)])
    return np.asarray(out, dtype=np.complex128)


def hankel_from_consecutive_moments(moments: ComplexArray, rows: int, cols: int) -> ComplexArray:
    if moments.size < rows + cols - 1:
        raise ValueError("not enough consecutive moments")
    return np.asarray([[moments[i + j] for j in range(cols)] for i in range(rows)], dtype=np.complex128)


def estimate_order_from_hankel(moments: ComplexArray, upper_bound: int, tolerance: float = 1e-10) -> tuple[
    int, Array]:
    H = hankel_from_consecutive_moments(moments, upper_bound, upper_bound)
    singular = np.linalg.svd(H, compute_uv=False)
    if singular[0] == 0:
        return 0, singular.astype(float)
    order = int(np.sum(singular > tolerance * singular[0]))
```

```python
    return order, singular.astype(float)


def recover_unknown_edges(
    measurements: dict[int, complex],
    background_degree_bound: int,
    edge_order_bound: int,
    known_order: int | None = None,
    rank_tolerance: float = 1e-10,
    project_roots_to_circle: bool = True,
) -> RecoveryResult:
    """Recover all edge locations and jumps from consecutive high Fourier modes."""
    n0 = int(background_degree_bound) + 1
    moment_indices = np.arange(n0, n0 + 2 * edge_order_bound, dtype=np.int64)
    moments = edge_moments_from_fourier(measurements, moment_indices)
    estimated_order, singular = estimate_order_from_hankel(moments, edge_order_bound, rank_tolerance)
    order = int(known_order if known_order is not None else estimated_order)
    if order < 1 or order > edge_order_bound:
        raise ValueError(f"invalid recovered edge order {order}")

    H = hankel_from_consecutive_moments(moments, order, order)
    b = moments[order : 2 * order]
    q_low = np.linalg.solve(H, -b)
    annihilator = np.r_[q_low, 1.0 + 0.0j]
    roots = np.roots(annihilator[::-1])
    if project_roots_to_circle:
        if np.min(np.abs(roots)) <= 1e-15:
            raise ValueError("annihilator produced a zero root")
        roots = roots / np.abs(roots)
    locations = np.mod(-np.angle(roots), TAU)
    sort_order = np.argsort(locations)
    locations = locations[sort_order]
    roots = roots[sort_order]

    # Use all available high moments for the amplitude solve.
    V = np.asarray(
        [[roots[j] ** int(n) for j in range(order)] for n in moment_indices],
        dtype=np.complex128,
    )
    jumps, _, _, _ = np.linalg.lstsq(V, moments, rcond=None)
    recurrence_values = []
    for start in range(0, moments.size - order):
        recurrence_values.append(np.dot(annihilator, moments[start : start + order + 1]))
    recurrence_residual = float(np.max(np.abs(recurrence_values))) if recurrence_values else 0.0
    amplitude_residual = float(np.linalg.norm(V @ jumps - moments))
    min_sep = float(np.min(pairwise_circular_gaps(locations)))
    hankel_condition = float(np.linalg.cond(H))
    vandermonde_condition = float(np.linalg.cond(V[:order, :]))
    return RecoveryResult(
        order=order,
        locations=locations,
        jumps=jumps,
        annihilator_ascending=annihilator,
        hankel_singular_values=singular,
        moment_indices=moment_indices,
        measured_moments=moments,
        recurrence_residual=recurrence_residual,
        amplitude_residual=amplitude_residual,
        min_root_separation=min_sep,
        hankel_condition=hankel_condition,
        vandermonde_condition=vandermonde_condition,
    )


def match_edge_models(true_edge: EdgeModel, recovered: RecoveryResult) -> dict[str, float | Array]:
    if true_edge.order != recovered.order:
        return {
            "location_error_max": float("inf"),
            "location_error_rmse": float("inf"),
            "jump_error_max": float("inf"),
            "jump_error_rmse": float("inf"),
```

```
            "matched_locations": recovered.locations,
            "matched_jumps": recovered.jumps,
        }
    cost = np.asarray(
        [[circular_distance(float(a), float(b)) for b in recovered.locations] for a in true_edge.locations],
        dtype=float,
    )
    rows, cols = linear_sum_assignment(cost)
    matched_locations = recovered.locations[cols]
    matched_jumps = recovered.jumps[cols]
    loc_errors = np.asarray([circular_distance(float(true_edge.locations[r]), float(matched_locations[i]))
     for i, r in enumerate(rows)])
    jump_errors = np.abs(true_edge.jumps[rows] - matched_jumps)
    return {
        "location_error_max": float(np.max(loc_errors)),
        "location_error_rmse": float(np.sqrt(np.mean(loc_errors**2))),
        "jump_error_max": float(np.max(jump_errors)),
        "jump_error_rmse": float(np.sqrt(np.mean(jump_errors**2))),
        "matched_locations": matched_locations,
        "matched_jumps": matched_jumps,
    }


def reconstruct_background(
    measurements: dict[int, complex],
    recovered_edge: EdgeModel,
    degree: int,
) -> BandlimitedBackground:
    coeffs: dict[int, complex] = {}
    for n in range(-degree, degree + 1):
        coeffs[n] = measurements[n] - recovered_edge.fourier(n)
    # Force the exact real-signal symmetry at roundoff scale.
    coeffs[0] = complex(float(np.real(coeffs[0])), 0.0)
    for n in range(1, degree + 1):
        avg = 0.5 * (coeffs[n] + np.conj(coeffs[-n]))
        coeffs[n] = avg
        coeffs[-n] = np.conj(avg)
    return BandlimitedBackground(degree, coeffs)


def raw_fourier_partial(measurements: dict[int, complex], x: Array, order: int) -> Array:
    out = np.zeros_like(x, dtype=np.complex128)
    for n in range(-order, order + 1):
        out += measurements.get(n, 0.0) * np.exp(1j * n * x)
    return np.real_if_close(out, tol=1000).astype(float)


def packet_resolved_reconstruction(
    measurements: dict[int, complex],
    recovered_edge: EdgeModel,
    x: Array,
    background_degree: int,
) -> Array:
    background = reconstruct_background(measurements, recovered_edge, background_degree)
    return background.values(x) + recovered_edge.values(x)


# ---------------------------------------------------------------------------
# Finite dictionary theorem
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class DictionaryProblem:
    background: Array
    dictionary: Array

    def __post_init__(self) -> None:
        B = np.asarray(self.background, dtype=float)
        D = np.asarray(self.dictionary, dtype=float)
        if B.ndim != 2 or D.ndim != 2 or B.shape[0] != D.shape[0]:
            raise ValueError("background and dictionary must share the observation dimension")
        object.__setattr__(self, "background", B)
```

```
        object.__setattr__(self, "dictionary", D)

    @property
    def projector(self) -> Array:
        q, _ = np.linalg.qr(self.background)
        return q @ q.T

    @property
    def residual_dictionary(self) -> Array:
        return (np.eye(self.background.shape[0]) - self.projector) @ self.dictionary


def restricted_separation_constant(matrix: Array, sparsity: int) -> float:
    """Exact finite support search for the smallest restricted singular value."""
    A = np.asarray(matrix, dtype=float)
    if sparsity < 1:
        return float("inf")
    best = float("inf")
    max_support = min(sparsity, A.shape[1])
    for size in range(1, max_support + 1):
        for support in combinations(range(A.shape[1]), size):
            sigma = np.linalg.svd(A[:, support], compute_uv=False)
            best = min(best, float(sigma[-1]))
    return best


def exhaustive_sparse_recovery(
    problem: DictionaryProblem,
    y: Array,
    sparsity: int,
) -> dict[str, object]:
    A = problem.residual_dictionary
    R = np.eye(problem.background.shape[0]) - problem.projector
    r = R @ np.asarray(y, dtype=float)
    best_residual = float("inf")
    best_alpha = np.zeros(A.shape[1], dtype=float)
    best_support: tuple[int, ...] = ()
    second_residual = float("inf")
    for size in range(0, sparsity + 1):
        for support in combinations(range(A.shape[1]), size):
            alpha = np.zeros(A.shape[1], dtype=float)
            if support:
                coeff, _, _, _ = np.linalg.lstsq(A[:, support], r, rcond=None)
                alpha[list(support)] = coeff
            residual = float(np.linalg.norm(A @ alpha - r))
            if residual < best_residual - 1e-14:
                second_residual = best_residual
                best_residual = residual
                best_alpha = alpha
                best_support = support
            elif residual < second_residual:
                second_residual = residual
    edge = problem.dictionary @ best_alpha
    smooth = problem.projector @ (y - edge)
    return {
        "support": best_support,
        "alpha": best_alpha,
        "edge": edge,
        "smooth": smooth,
        "reconstruction": smooth + edge,
        "residual": best_residual,
        "second_residual": second_residual,
        "gap": second_residual - best_residual,
    }


# ---------------------------------------------------------------------------
# Multi-channel recovery
# ---------------------------------------------------------------------------
def recover_shared_edges_multichannel(
    measurements_by_channel: Sequence[dict[int, complex]],
```

```
    background_degree_bound: int,
    edge_order_bound: int,
    known_order: int,
) -> RecoveryResult:
    n0 = background_degree_bound + 1
    moment_indices = np.arange(n0, n0 + 2 * edge_order_bound, dtype=np.int64)
    channel_moments = [edge_moments_from_fourier(m, moment_indices) for m in measurements_by_channel]
    order = known_order
    H_blocks = []
    b_blocks = []
    for moments in channel_moments:
        H_blocks.append(hankel_from_consecutive_moments(moments, order, order))
        b_blocks.append(moments[order : 2 * order])
    H_stack = np.vstack(H_blocks)
    b_stack = np.concatenate(b_blocks)
    q_low, _, _, _ = np.linalg.lstsq(H_stack, -b_stack, rcond=None)
    annihilator = np.r_[q_low, 1.0 + 0.0j]
    roots = np.roots(annihilator[::-1])
    roots = roots / np.abs(roots)
    locations = np.mod(-np.angle(roots), TAU)
    order_idx = np.argsort(locations)
    locations = locations[order_idx]
    roots = roots[order_idx]
    # Aggregate amplitudes only for diagnostics. The location theorem is the key.
    jumps_aggregate = np.zeros(order, dtype=np.complex128)
    residuals = []
    V = np.asarray([[roots[j] ** int(n) for j in range(order)] for n in moment_indices], dtype=np.complex128)
    for moments in channel_moments:
        amp, _, _, _ = np.linalg.lstsq(V, moments, rcond=None)
        jumps_aggregate += amp
        residuals.append(float(np.linalg.norm(V @ amp - moments)))
    rec_residual = 0.0
    for moments in channel_moments:
        for start in range(moments.size - order):
            rec_residual = max(rec_residual, float(abs(np.dot(annihilator, moments[start : start + order +
 1]))))
    H_square = H_stack[:order, :]
    return RecoveryResult(
        order=order,
        locations=locations,
        jumps=jumps_aggregate,
        annihilator_ascending=annihilator,
        hankel_singular_values=np.linalg.svd(H_stack, compute_uv=False).astype(float),
        moment_indices=moment_indices,
        measured_moments=np.concatenate(channel_moments),
        recurrence_residual=rec_residual,
        amplitude_residual=max(residuals),
        min_root_separation=float(np.min(pairwise_circular_gaps(locations))),
        hankel_condition=float(np.linalg.cond(H_square)),
        vandermonde_condition=float(np.linalg.cond(V[:order, :])),
    )


# ------------------------------------------------------------------------------
# Piecewise-polynomial recurrence check
# ------------------------------------------------------------------------------
def exponential_polynomial_moments(
    roots: ComplexArray,
    polynomial_coefficients: Sequence[Sequence[complex]],
    indices: Sequence[int],
) -> ComplexArray:
    out = []
    for n in indices:
        value = 0.0 + 0.0j
        for root, coeffs in zip(roots, polynomial_coefficients, strict=True):
            p = sum(complex(c) * (n**r) for r, c in enumerate(coeffs))
            value += p * (root**n)
        out.append(value)
    return np.asarray(out, dtype=np.complex128)


```

```
def repeated_root_annihilator(roots: ComplexArray, multiplicity: int) -> ComplexArray:
    poly_desc = np.asarray([1.0 + 0.0j])
    for root in roots:
        for _ in range(multiplicity):
            poly_desc = np.convolve(poly_desc, np.asarray([1.0 + 0.0j, -root], dtype=np.complex128))
    return poly_desc[::-1]


# -----------------------------------------------------------------------------
# Diagnostics and experiments
# -----------------------------------------------------------------------------
def location_error(true_locations: Array, recovered_locations: Array) -> float:
    if true_locations.size != recovered_locations.size:
        return float("inf")
    cost = np.asarray(
        [[circular_distance(float(a), float(b)) for b in recovered_locations] for a in true_locations],
        dtype=float,
    )
    rows, cols = linear_sum_assignment(cost)
    return float(np.max(cost[rows, cols]))


def add_fourier_noise(
    measurements: dict[int, complex],
    indices: Iterable[int],
    sigma: float,
    rng: np.random.Generator,
) -> dict[int, complex]:
    out = dict(measurements)
    for n in indices:
        noise = sigma * (rng.normal() + 1j * rng.normal()) / math.sqrt(2.0)
        out[int(n)] = out[int(n)] + noise
    return out


def exact_symbolic_example() -> dict[str, str]:
    """An exact four-edge roots-of-unity example checked in SymPy."""
    I = sp.I
    roots = [sp.Integer(1), -sp.Integer(1), I, -I]
    jumps = [sp.Integer(2), sp.Integer(-1), sp.Integer(3), sp.Integer(-4)]
    n0 = 3
    moments = [sp.simplify(sum(a * z**n for a, z in zip(jumps, roots, strict=True))) for n in range(n0, n0 +
     8)]
    H = sp.Matrix([[moments[i + j] for j in range(4)] for i in range(4)])
    b = sp.Matrix([moments[i + 4] for i in range(4)])
    q = H.LUsolve(-b)
    Q = sp.expand(sum(q[r] * sp.Symbol("z") ** r for r in range(4)) + sp.Symbol("z") ** 4)
    expected = sp.expand((sp.Symbol("z") - 1) * (sp.Symbol("z") + 1) * (sp.Symbol("z") - I) * (sp.Symbol("z")
      + I))
    return {
        "hankel_determinant": str(sp.simplify(H.det())),
        "annihilator": str(Q),
        "expected": str(expected),
        "difference": str(sp.expand(Q - expected)),
    }


def run_exact_examples() -> tuple[list[dict[str, float | int]], dict[str, object]]:
    rows: list[dict[str, float | int]] = []
    rng = np.random.default_rng(20260729)
    cases = [
        (3, 2, 4, 0.85),
        (4, 4, 6, 0.55),
        (5, 6, 8, 0.42),
        (6, 8, 10, 0.34),
    ]
    representative: dict[str, object] = {}
    for index, (K, L, Lmax, separation) in enumerate(cases):
        edge = generate_edge_model(L, rng, min_separation=separation)
        background = generate_background(K, rng)
        signal = UniversalSignal(background, edge)
```

```
        indices = range(-K, K + 2 * Lmax + 2)
        measurements = fourier_measurements(signal, indices)
        result = recover_unknown_edges(measurements, K, Lmax)
        matched = match_edge_models(edge, result)
        recovered_edge = result.edge_model()
        recovered_background = reconstruct_background(measurements, recovered_edge, K)
        x = np.linspace(0.0, TAU, 4096, endpoint=False)
        reconstruction = recovered_background.values(x) + recovered_edge.values(x)
        error = float(np.max(np.abs(reconstruction - signal.values(x))))
        rows.append(
            {
                "case": index + 1,
                "degree": K,
                "true_edges": L,
                "recovered_edges": result.order,
                "location_error": float(matched["location_error_max"]),
                "jump_error": float(matched["jump_error_max"]),
                "reconstruction_error": error,
                "hankel_condition": result.hankel_condition,
                "vandermonde_condition": result.vandermonde_condition,
            }
        )
        if index == 1:
            representative = {
                "edge": edge,
                "background": background,
                "signal": signal,
                "measurements": measurements,
                "result": result,
                "recovered_edge": recovered_edge,
                "recovered_background": recovered_background,
            }
    return rows, representative


def run_noise_experiment() -> list[dict[str, float]]:
    rng = np.random.default_rng(314159)
    sigmas = [0.0, 1e-10, 1e-8, 1e-6, 1e-4, 5e-4, 1e-3]
    separations = [0.22, 0.35, 0.55, 0.80]
    rows: list[dict[str, float]] = []
    K = 4
    L = 4
    Lmax = 6
    trials = 32
    n0 = K + 1
    high_indices = list(range(n0, n0 + 2 * Lmax))
    all_indices = list(range(-K, K + 2 * Lmax + 2))
    for sep in separations:
        for sigma in sigmas:
            loc_errors = []
            rec_errors = []
            successes = 0
            conditions = []
            for _ in range(trials):
                edge = generate_edge_model(L, rng, min_separation=sep)
                background = generate_background(K, rng)
                signal = UniversalSignal(background, edge)
                measurements = fourier_measurements(signal, all_indices)
                noisy = add_fourier_noise(measurements, high_indices, sigma, rng)
                try:
                    result = recover_unknown_edges(noisy, K, Lmax, known_order=L, rank_tolerance=1e-8)
                    loc = location_error(edge.locations, result.locations)
                    loc_errors.append(loc)
                    conditions.append(result.hankel_condition)
                    successes += int(loc < 0.02)
                    # The smooth coefficients are not noised in this controlled study.
                    recovered_edge = result.edge_model()
                    recovered_background = reconstruct_background(noisy, recovered_edge, K)
                    x = np.linspace(0.0, TAU, 2048, endpoint=False)
                    rec_errors.append(float(np.sqrt(np.mean((signal.values(x) - (recovered_background.values(
    x) + recovered_edge.values(x))) ** 2))))
```

```
                except Exception:
                    loc_errors.append(PI)
                    rec_errors.append(1.0)
                    conditions.append(float("inf"))
            finite_conditions = [c for c in conditions if np.isfinite(c)]
            rows.append(
                {
                    "separation": sep,
                    "sigma": sigma,
                    "success_rate": successes / trials,
                    "median_location_error": float(np.median(loc_errors)),
                    "mean_location_error": float(np.mean(loc_errors)),
                    "median_reconstruction_rmse": float(np.median(rec_errors)),
                    "median_hankel_condition": float(np.median(finite_conditions)) if finite_conditions else
     float("inf"),
                }
            )
    return rows


def run_close_edge_experiment() -> list[dict[str, float]]:
    K = 2
    L = 4
    Lmax = 4
    base = np.asarray([0.4, 2.4, 4.4, 5.6])
    jumps = np.asarray([1.0, -0.8, 0.5, -0.7])
    separations = np.geomspace(0.8, 0.015, 28)
    rows = []
    rng = np.random.default_rng(99)
    background = generate_background(K, rng)
    for delta in separations:
        tau = base.copy()
        tau[1] = tau[0] + delta
        tau = np.sort(np.mod(tau, TAU))
        edge = EdgeModel(tau, jumps)
        signal = UniversalSignal(background, edge)
        indices = list(range(-K, K + 2 * Lmax + 2))
        measurements = fourier_measurements(signal, indices)
        result = recover_unknown_edges(measurements, K, Lmax, known_order=L)
        rows.append(
            {
                "minimum_separation": edge.min_separation,
                "hankel_condition": result.hankel_condition,
                "vandermonde_condition": result.vandermonde_condition,
                "location_error": location_error(edge.locations, result.locations),
            }
        )
    return rows


def run_runtime_experiment() -> list[dict[str, float | int]]:
    rng = np.random.default_rng(404)
    rows = []
    for L in range(2, 13):
        K = 4
        Lmax = L + 2
        sep = min(0.25, 0.8 * TAU / L)
        edge = generate_edge_model(L, rng, min_separation=sep)
        background = generate_background(K, rng)
        signal = UniversalSignal(background, edge)
        indices = list(range(-K, K + 2 * Lmax + 2))
        measurements = fourier_measurements(signal, indices)
        repeats = 40
        start = time.perf_counter()
        for _ in range(repeats):
            recover_unknown_edges(measurements, K, Lmax, known_order=L)
        elapsed = (time.perf_counter() - start) / repeats
        rows.append({"edge_order": L, "seconds": elapsed})
    return rows


```

```
def build_dictionary_example() -> tuple[DictionaryProblem, Array, Array, dict[str, object], float]:
    n = 24
    x = np.linspace(0.0, TAU, n, endpoint=False)
    B = np.column_stack([np.ones(n), np.cos(x), np.sin(x), np.cos(2 * x), np.sin(2 * x)])
    atoms = []
    for start in range(0, n, 3):
        mask = np.zeros(n)
        mask[start : min(start + 4, n)] = 1.0
        atoms.append(mask)
    D = np.column_stack(atoms)
    problem = DictionaryProblem(B, D)
    alpha = np.zeros(D.shape[1])
    alpha[1] = 1.2
    alpha[5] = -0.7
    theta = np.asarray([0.2, 0.5, -0.25, 0.1, 0.15])
    y = B @ theta + D @ alpha
    recovered = exhaustive_sparse_recovery(problem, y, sparsity=2)
    gamma = restricted_separation_constant(problem.residual_dictionary, sparsity=4)
    return problem, y, alpha, recovered, gamma


def build_multichannel_example() -> dict[str, object]:
    rng = np.random.default_rng(909)
    locations = np.asarray([0.45, 1.70, 3.05, 4.30])
    jumps1 = np.asarray([1.0, -0.7, 0.4, -0.7])
    jumps2 = np.asarray([0.0, 0.8, -1.1, 0.3])
    # The first edge is invisible in channel 2, but visible in channel 1.
    edge1 = EdgeModel(locations, jumps1)
    # Avoid a zero jump in the constructor by using the moment model directly for channel 2.
    K = 3
    L = 4
    Lmax = 6
    background1 = generate_background(K, rng)
    background2 = generate_background(K, rng)
    indices = list(range(-K, K + 2 * Lmax + 2))
    m1 = {}
    m2 = {}
    for n in indices:
        edge2_fourier = 0.0 if n == 0 else np.sum(jumps2 * np.exp(-1j * n * locations)) / (2j * PI * n)
        m1[n] = background1.fourier(n) + edge1.fourier(n)
        m2[n] = background2.fourier(n) + edge2_fourier
    result = recover_shared_edges_multichannel([m1, m2], K, Lmax, known_order=L)
    return {
        "locations": locations,
        "jumps1": jumps1,
        "jumps2": jumps2,
        "measurements1": m1,
        "measurements2": m2,
        "result": result,
    }




def build_2d_rectangle_example() -> dict[str, object]:
    height = 18
    width = 18
    yy, xx = np.mgrid[0:height, 0:width]
    xn = 2.0 * PI * xx.ravel() / width
    yn = 2.0 * PI * yy.ravel() / height
    background_design = np.column_stack(
        [
            np.ones(height * width),
            np.cos(xn),
            np.sin(xn),
            np.cos(yn),
            np.sin(yn),
            np.cos(xn + yn),
        ]
    )
    q, _ = np.linalg.qr(background_design)
    candidates: list[Array] = []
```

```
    metadata: list[tuple[int, int, int, int]] = []
    for rectangle_height in range(3, 8):
        for rectangle_width in range(3, 8):
            for top in range(0, height - rectangle_height + 1):
                for left in range(0, width - rectangle_width + 1):
                    mask = np.zeros((height, width), dtype=float)
                    mask[top : top + rectangle_height, left : left + rectangle_width] = 1.0
                    candidates.append(mask.ravel())
                    metadata.append((top, left, rectangle_height, rectangle_width))
    dictionary = np.column_stack(candidates)
    residual_dictionary = dictionary - q @ (q.T @ dictionary)
    true_metadata = (6, 8, 5, 6)
    true_index = metadata.index(true_metadata)
    amplitude = 1.35
    theta = np.asarray([0.2, 0.42, -0.18, 0.31, 0.12, -0.16])
    smooth = background_design @ theta
    edge = amplitude * dictionary[:, true_index]
    observed = smooth + edge
    residual_observed = observed - q @ (q.T @ observed)
    dots = residual_dictionary.T @ residual_observed
    norms = np.sum(residual_dictionary**2, axis=0)
    amplitudes = dots / norms
    residuals = (
        float(residual_observed @ residual_observed)
        - 2.0 * amplitudes * dots
        + amplitudes**2 * norms
    )
    best = int(np.argmin(residuals))
    recovered_edge = amplitudes[best] * dictionary[:, best]
    recovered_smooth = q @ (q.T @ (observed - recovered_edge))
    reconstruction = recovered_smooth + recovered_edge
    return {
        "height": height,
        "width": width,
        "metadata": metadata,
        "true_metadata": true_metadata,
        "true_index": true_index,
        "best_index": best,
        "best_metadata": metadata[best],
        "amplitude": amplitude,
        "recovered_amplitude": float(amplitudes[best]),
        "smooth": smooth.reshape(height, width),
        "edge": edge.reshape(height, width),
        "observed": observed.reshape(height, width),
        "recovered_edge": recovered_edge.reshape(height, width),
        "recovered_smooth": recovered_smooth.reshape(height, width),
        "reconstruction": reconstruction.reshape(height, width),
        "residual": (reconstruction - observed).reshape(height, width),
        "objective": float(residuals[best]),
        "second_objective": float(np.partition(residuals, 1)[1]),
    }

def run_tests(
    exact_rows: list[dict[str, float | int]],
    representative: dict[str, object],
    dictionary_data: tuple[DictionaryProblem, Array, Array, dict[str, object], float],
    multichannel: dict[str, object],
    rectangle_2d: dict[str, object],
) -> list[dict[str, object]]:
    tests: list[dict[str, object]] = []

    def record(name: str, passed: bool, detail: str) -> None:
        tests.append({"name": name, "passed": bool(passed), "detail": detail})
        if not passed:
            raise AssertionError(f"{name}: {detail}")

    symbolic = exact_symbolic_example()
    record("symbolic annihilator", symbolic["difference"] == "0", symbolic["annihilator"])

    edge: EdgeModel = representative["edge"]  # type: ignore[assignment]
    signal: UniversalSignal = representative["signal"]  # type: ignore[assignment]
```

```
    measurements: dict[int, complex] = representative["measurements"]  # type: ignore[assignment]
    result: RecoveryResult = representative["result"]  # type: ignore[assignment]
    recovered_edge: EdgeModel = representative["recovered_edge"]  # type: ignore[assignment]
    recovered_background: BandlimitedBackground = representative["recovered_background"]  # type: ignore[
     assignment]

    n_test = np.arange(5, 17)
    direct = edge.moments(n_test)
    from_fourier = edge_moments_from_fourier(measurements, n_test)
    record("distributional Fourier identity", float(np.max(np.abs(direct - from_fourier))) < 1e-11, f"
     residual={np.max(np.abs(direct-from_fourier)):.3e}")

    record("unknown order", result.order == edge.order, f"true={edge.order}, recovered={result.order}")
    match = match_edge_models(edge, result)
    record("off-grid locations", float(match["location_error_max"]) < 1e-9, f"error={float(match['
     location_error_max']):.3e}")
    record("jump amplitudes", float(match["jump_error_max"]) < 1e-8, f"error={float(match['jump_error_max'])
     :.3e}")
    record("annihilating recurrence", result.recurrence_residual < 1e-8, f"residual={result.
     recurrence_residual:.3e}")
    record("Vandermonde amplitude solve", result.amplitude_residual < 1e-8, f"residual={result.
     amplitude_residual:.3e}")

    x = np.linspace(0.0, TAU, 8192, endpoint=False)
    reconstruction = recovered_background.values(x) + recovered_edge.values(x)
    error = float(np.max(np.abs(reconstruction - signal.values(x))))
    record("full signal reconstruction", error < 1e-8, f"error={error:.3e}")

    K = signal.background.degree
    raw = raw_fourier_partial(measurements, x, order=K)
    resolved = packet_resolved_reconstruction(measurements, recovered_edge, x, K)
    record("Gibbs-free resolved synthesis", float(np.max(np.abs(resolved - signal.values(x)))) < 1e-8, f"
     raw_rmse={np.sqrt(np.mean((raw-signal.values(x))**2)):.3e}")

    # Rank factorization and exact rows.
    record("all exact random cases", max(float(row["reconstruction_error"]) for row in exact_rows) < 1e-7, f"
     cases={len(exact_rows)}")

    problem, y, alpha, recovered, gamma = dictionary_data
    record("finite dictionary support", tuple(np.flatnonzero(alpha)) == tuple(recovered["support"]), f"
     support={recovered['support']}")
    record("finite dictionary synthesis", float(np.max(np.abs(recovered["reconstruction"] - y))) < 1e-10, f"
     residual={recovered['residual']:.3e}")
    record("positive restricted separation", gamma > 1e-6, f"gamma4={gamma:.3e}")

    # A deterministic noisy stability check for the finite dictionary theorem.
    rng = np.random.default_rng(17)
    eta = 1e-4 * rng.normal(size=y.size)
    noisy = y + eta
    recovered_noisy = exhaustive_sparse_recovery(problem, noisy, sparsity=2)
    coeff_error = float(np.linalg.norm(recovered_noisy["alpha"] - alpha))
    bound = 2.0 * float(np.linalg.norm(eta)) / gamma
    record("finite dictionary stability", coeff_error <= bound + 1e-10, f"error={coeff_error:.3e}, bound={
     bound:.3e}")

    multi_result: RecoveryResult = multichannel["result"]  # type: ignore[assignment]
    multi_error = location_error(np.asarray(multichannel["locations"]), multi_result.locations)
    record("shared-edge multichannel recovery", multi_error < 1e-8, f"error={multi_error:.3e}")

    # Piecewise-polynomial repeated-root recurrence.
    roots = np.exp(-1j * np.asarray([0.7, 2.2]))
    coeffs = [[1.0, 0.3], [-0.4, 0.8]]
    indices = np.arange(1, 20)
    moments = exponential_polynomial_moments(roots, coeffs, indices)
    annihilator = repeated_root_annihilator(roots, multiplicity=2)
    residual = max(abs(np.dot(annihilator, moments[i : i + annihilator.size])) for i in range(moments.size -
     annihilator.size + 1))
    record("piecewise-polynomial confluent recurrence", float(residual) < 1e-9, f"residual={float(residual)
     :.3e}")

    # Concrete impossibility witness: a sub-grid interval missed by all samples.
```

```
    sample_x = np.linspace(0.0, 1.0, 8, endpoint=False)
    witness = ((sample_x > 0.13) & (sample_x < 0.20)).astype(float)
    record("finite-sampling impossibility witness", float(np.max(np.abs(witness))) == 0.0, "nonzero interval can lie between sample points")

    # No uniform stability as two opposite jumps collide.
    n = np.arange(1, 13)
    delta1 = 1e-2
    delta2 = 5e-3
    m1 = 1.0 - np.exp(-1j * n * delta1)
    m2 = 1.0 - np.exp(-1j * n * delta2)
    record("collision instability", float(np.linalg.norm(m2)) < float(np.linalg.norm(m1)), "moment norm shrinks with edge separation")

    record(
        "two-dimensional rectangle recovery",
        rectangle_2d["best_metadata"] == rectangle_2d["true_metadata"]
        and abs(float(rectangle_2d["recovered_amplitude"]) - float(rectangle_2d["amplitude"])) < 1e-10
        and float(np.max(np.abs(rectangle_2d["residual"]))) < 1e-10,
        f"true={rectangle_2d['true_metadata']}, recovered={rectangle_2d['best_metadata']}",
    )

    return tests


# ---------------------------------------------------------------------------
# Figure generation
# ---------------------------------------------------------------------------
def generate_figures(
    base: Path,
    exact_rows: list[dict[str, float | int]],
    representative: dict[str, object],
    noise_rows: list[dict[str, float]],
    close_rows: list[dict[str, float]],
    runtime_rows: list[dict[str, float | int]],
    dictionary_data: tuple[DictionaryProblem, Array, Array, dict[str, object], float],
    multichannel: dict[str, object],
    rectangle_2d: dict[str, object],
    tests: list[dict[str, object]],
) -> None:
    out = base / "figures"
    edge: EdgeModel = representative["edge"]  # type: ignore[assignment]
    background: BandlimitedBackground = representative["background"]  # type: ignore[assignment]
    signal: UniversalSignal = representative["signal"]  # type: ignore[assignment]
    measurements: dict[int, complex] = representative["measurements"]  # type: ignore[assignment]
    result: RecoveryResult = representative["result"]  # type: ignore[assignment]
    recovered_edge: EdgeModel = representative["recovered_edge"]  # type: ignore[assignment]
    recovered_background: BandlimitedBackground = representative["recovered_background"]  # type: ignore[assignment]

    # 01 Pipeline.
    fig, ax = diagram_axes((13.0, 5.8))
    stages = [
        ("finite Fourier data", BLUE),
        ("remove the smooth band", TEAL),
        ("form edge moments", GREEN),
        ("build the Hankel packet", ORANGE),
        ("recover the annihilator", MAGENTA),
        ("find edges and jumps", PURPLE),
        ("synthesize without Gibbs", RED),
    ]
    xs = np.linspace(0.02, 0.84, len(stages))
    for i, ((label, color), xpos) in enumerate(zip(stages, xs, strict=True)):
        draw_box(ax, (xpos, 0.42), 0.13, 0.20, label, face=color + "20", edge=color, fontsize=10.5)
        if i + 1 < len(stages):
            draw_arrow(ax, (xpos + 0.13, 0.52), (xs[i + 1] - 0.006, 0.52))
    ax.text(0.5, 0.88, "Unknown-edge recovery as one closed projector-resolved chain", ha="center", fontsize=22, weight="bold")
    ax.text(0.5, 0.12, "The word universal refers to every signal in the declared finite-rate class, not to arbitrary functions without assumptions.", ha="center", color=MUTED, fontsize=11.5)
    save_figure(fig, out, "01_closed_pipeline")
```

```
    # 02 Impossibility witness.
    fig, axes = plt.subplots(2, 1, figsize=(12.4, 6.2), sharex=True)
    xx = np.linspace(0.0, 1.0, 1200)
    samples = np.linspace(0.0, 1.0, 8, endpoint=False)
    hidden = ((xx > 0.13) & (xx < 0.20)).astype(float)
    axes[0].plot(xx, np.zeros_like(xx), color=INK, lw=2)
    axes[0].scatter(samples, np.zeros_like(samples), color=BLUE, zorder=4, label="samples")
    axes[0].set_title("Signal A: no edge", loc="left", weight="bold")
    axes[1].plot(xx, hidden, color=RED, lw=2)
    axes[1].scatter(samples, np.zeros_like(samples), color=BLUE, zorder=4, label="the same samples")
    axes[1].set_title("Signal B: a hidden interval between sample points", loc="left", weight="bold")
    for axr in axes:
        axr.set_ylim(-0.15, 1.2)
        axr.grid(alpha=.16)
        axr.legend(loc="upper right")
    axes[-1].set_xlabel("coordinate")
    fig.suptitle("No method can recover arbitrary edges from a noninjective measurement map", fontsize=17,
     weight="bold")
    fig.tight_layout(rect=[0, 0, 1, .94])
    save_figure(fig, out, "02_unrestricted_impossibility")

    # 03 Decomposition ambiguity.
    fig, ax = diagram_axes((12.6, 5.8))
    draw_box(ax, (.06, .55), .22, .18, "observed object\n$f$", face=BLUE+"20", edge=BLUE, fontsize=16)
    draw_box(ax, (.40, .68), .23, .16, "smooth packet\n$s+h$", face=TEAL+"20", edge=TEAL, fontsize=14)
    draw_box(ax, (.40, .32), .23, .16, "edge packet\n$j-h$", face=ORANGE+"20", edge=ORANGE, fontsize=14)
    draw_box(ax, (.76, .50), .18, .20, "same sum\n$(s+h)+(j-h)=f$", face=MAGENTA+"18", edge=MAGENTA, fontsize
     =13)
    draw_arrow(ax, (.28,.64), (.40,.76), color=TEAL)
    draw_arrow(ax, (.28,.64), (.40,.40), color=ORANGE)
    draw_arrow(ax, (.63,.76), (.76,.62), color=TEAL)
    draw_arrow(ax, (.63,.40), (.76,.57), color=ORANGE)
    ax.text(.5,.13,r"If $h\in\mathcal{S}\cap\mathcal{J}$ is nonzero, the smooth-edge split is not
     identifiable.",ha="center",fontsize=15)
    ax.text(.5,.08,"A recovery theorem must make the two packets distinguishable after measurement.",ha="
     center",color=MUTED,fontsize=11)
    save_figure(fig,out,"03_decomposition_ambiguity")

    # 04 Declared model.
    x = np.linspace(0.0, TAU, 4096, endpoint=False)
    fig, axes = plt.subplots(3, 1, figsize=(12.5, 7.0), sharex=True)
    axes[0].plot(x, background.values(x), color=TEAL, lw=2)
    axes[0].set_title("Unknown bandlimited background", loc="left", weight="bold")
    axes[1].plot(x, edge.values(x), color=ORANGE, lw=2)
    for t in edge.locations:
        axes[1].axvline(t, color=RED, lw=.8, alpha=.6)
    axes[1].set_title(f"Unknown piecewise-constant edge packet with {edge.order} off-grid jumps", loc="left",
      weight="bold")
    axes[2].plot(x, signal.values(x), color=INK, lw=2)
    axes[2].set_title("Observed sum", loc="left", weight="bold")
    for axr in axes:
        axr.grid(alpha=.15)
        axr.set_ylabel("amplitude")
    axes[-1].set_xlabel("periodic coordinate")
    fig.tight_layout()
    save_figure(fig,out,"04_declared_model")

    # 05 Distributional derivative atoms.
    fig, ax = plt.subplots(figsize=(12.4, 5.3))
    ax.plot(x, edge.values(x), color=ORANGE, lw=2.2, label="edge packet")
    y0 = np.min(edge.values(x)) - 0.55
    for i, (t, a) in enumerate(zip(edge.locations, edge.jumps, strict=True)):
        ax.arrow(t, y0, 0, a * .35, width=.008, head_width=.08, head_length=.10, color=PALETTE[i % len(
     PALETTE)], length_includes_head=True)
        ax.text(t, y0 - .12, rf"$a_{{{i+1}}}$", ha="center", color=PALETTE[i % len(PALETTE)], weight="bold")
    ax.axhline(y0, color=GRID, lw=1)
    ax.set_title(r"The derivative packet is finite: $j'=\sum_\ell a_\ell\delta_{\tau_\ell}$", weight="bold")
    ax.set_xlabel("coordinate")
    ax.set_ylabel("amplitude")
    ax.grid(alpha=.15)
```

```
    ax.legend(loc="upper right")
    save_figure(fig,out,"05_distributional_derivative")

    # 06 Fourier separation.
    ns = np.arange(-28, 29)
    coeff = np.asarray([signal.fourier(int(n)) for n in ns])
    fig, ax = plt.subplots(figsize=(12.3, 5.4))
    ax.stem(ns, np.abs(coeff), linefmt=BLUE, markerfmt="o", basefmt=" ")
    ax.axvspan(-background.degree, background.degree, color=TEAL, alpha=.16, label="smooth band")
    ax.axvline(background.degree + 0.5, color=RED, ls="--", lw=1.3)
    ax.axvline(-background.degree - 0.5, color=RED, ls="--", lw=1.3)
    ax.set_yscale("log")
    ax.set_xlabel("Fourier index")
    ax.set_ylabel(r"$|\widehat f(n)|$")
    ax.set_title("Beyond the smooth band, every coefficient belongs to the edge packet", weight="bold")
    ax.grid(alpha=.18, which="both")
    ax.legend(loc="upper right")
    save_figure(fig,out,"06_fourier_packet_separation")

    # 07 Edge moments.
    moment_indices = result.moment_indices
    moments = result.measured_moments
    fig, axes = plt.subplots(2, 1, figsize=(12.2, 6.2), sharex=True)
    axes[0].stem(moment_indices, moments.real, linefmt=BLUE, markerfmt="o", basefmt=" ")
    axes[0].set_ylabel("real part")
    axes[1].stem(moment_indices, moments.imag, linefmt=MAGENTA, markerfmt="o", basefmt=" ")
    axes[1].set_ylabel("imaginary part")
    axes[1].set_xlabel("moment index n")
    for axr in axes:
        axr.grid(alpha=.16)
    fig.suptitle(r"Edge moments $m_n=2\pi i n\widehat f(n)=\sum_\ell a_\ell z_\ell^n$", fontsize=17, weight="
     bold")
    fig.tight_layout(rect=[0,0,1,.93])
    save_figure(fig,out,"07_edge_moments")

    # 08 Hankel matrix.
    H = hankel_from_consecutive_moments(moments, result.order, result.order)
    fig, axes = plt.subplots(1, 2, figsize=(12.2, 4.9))
    im0 = axes[0].imshow(H.real, cmap="coolwarm")
    axes[0].set_title("real part")
    im1 = axes[1].imshow(H.imag, cmap="coolwarm")
    axes[1].set_title("imaginary part")
    for axr in axes:
        axr.set_xlabel("column")
        axr.set_ylabel("row")
    fig.colorbar(im0, ax=axes[0], fraction=.046)
    fig.colorbar(im1, ax=axes[1], fraction=.046)
    fig.suptitle("The Hankel packet factors through a Vandermonde matrix and the unknown jumps", fontsize
     =16.5, weight="bold")
    fig.tight_layout(rect=[0,0,1,.92])
    save_figure(fig,out,"08_hankel_packet")

    # 09 Hankel singular values and unknown order.
    fig, ax = plt.subplots(figsize=(10.5, 5.1))
    sv = result.hankel_singular_values
    ax.semilogy(np.arange(1, len(sv)+1), sv, marker="o", color=BLUE, lw=2)
    ax.axvline(result.order + .5, color=RED, ls="--", label=f"rank = {result.order}")
    ax.set_xlabel("singular-value index")
    ax.set_ylabel("singular value")
    ax.set_title("The number of unknown edges is the exact Hankel rank", weight="bold")
    ax.grid(alpha=.18, which="both")
    ax.legend(loc="upper right")
    save_figure(fig,out,"09_unknown_order_rank")

    # 10 Annihilator roots.
    fig, ax = plt.subplots(figsize=(6.4, 6.4))
    circle = plt.Circle((0,0),1.0,fill=False,color=GRID,lw=2)
    ax.add_patch(circle)
    roots = np.exp(-1j * result.locations)
    ax.scatter(roots.real, roots.imag, s=90, color=RED, label="recovered roots")
    true_roots = np.exp(-1j * edge.locations)
```

```
    ax.scatter(true_roots.real, true_roots.imag, s=180, facecolors="none", edgecolors=INK, linewidths=1.7, label="true roots")
    for i,z in enumerate(roots):
        ax.text(z.real*1.10,z.imag*1.10,str(i+1),ha="center",va="center",weight="bold")
    ax.set_aspect("equal")
    ax.set_xlim(-1.25,1.25)
    ax.set_ylim(-1.25,1.25)
    ax.axhline(0,color=GRID,lw=.8)
    ax.axvline(0,color=GRID,lw=.8)
    ax.set_title("Roots of the annihilating polynomial encode the edge locations", weight="bold")
    ax.legend(loc="lower center", bbox_to_anchor=(.5,-.13), ncol=2)
    save_figure(fig,out,"10_annihilator_roots")

    # 11 Location comparison.
    fig, ax = plt.subplots(figsize=(12.0, 3.9))
    ax.hlines(0.55, 0, TAU, color=GRID, lw=2)
    ax.hlines(0.20, 0, TAU, color=GRID, lw=2)
    ax.scatter(edge.locations, np.full(edge.order,.55), s=90, color=INK, label="true")
    ax.scatter(result.locations, np.full(result.order,.20), s=90, marker="x", color=RED, label="recovered")
    for t in edge.locations:
        ax.plot([t,t],[.20,.55],color=TEAL,lw=1,alpha=.7)
    ax.set_xlim(0,TAU)
    ax.set_yticks([.20,.55],["recovered","true"])
    ax.set_xlabel("periodic coordinate")
    ax.set_title("All off-grid edge locations are recovered from finitely many Fourier moments", weight="bold")
    ax.legend(loc="upper center", ncol=2)
    ax.grid(alpha=.12,axis="x")
    save_figure(fig,out,"11_edge_location_recovery")

    # 12 Jump amplitudes.
    match = match_edge_models(edge,result)
    matched_jumps = np.real(np.asarray(match["matched_jumps"]))
    p = np.arange(edge.order)
    fig, ax = plt.subplots(figsize=(11.0,5.0))
    ax.bar(p-.18,edge.jumps,width=.36,color=TEAL,label="true")
    ax.bar(p+.18,matched_jumps,width=.36,color=ORANGE,label="recovered")
    ax.axhline(0,color=INK,lw=.8)
    ax.set_xticks(p,[f"edge {i+1}" for i in p])
    ax.set_ylabel("jump amplitude")
    ax.set_title("The Vandermonde solve recovers every jump amplitude", weight="bold")
    ax.grid(alpha=.15,axis="y")
    ax.legend(loc="upper right")
    save_figure(fig,out,"12_jump_amplitudes")

    # 13 Recovered packets.
    fig, axes = plt.subplots(3,1,figsize=(12.5,7.0),sharex=True)
    axes[0].plot(x,background.values(x),color=TEAL,lw=2,label="true")
    axes[0].plot(x,recovered_background.values(x),color=RED,lw=1.2,ls="--",label="recovered")
    axes[0].set_title("background packet",loc="left",weight="bold")
    axes[1].plot(x,edge.values(x),color=ORANGE,lw=2,label="true")
    axes[1].plot(x,recovered_edge.values(x),color=PURPLE,lw=1.2,ls="--",label="recovered")
    axes[1].set_title("edge packet",loc="left",weight="bold")
    axes[2].plot(x,signal.values(x),color=INK,lw=2,label="true")
    axes[2].plot(x,recovered_background.values(x)+recovered_edge.values(x),color=RED,lw=1.0,ls="--",label="recovered")
    axes[2].set_title("complete signal",loc="left",weight="bold")
    for axr in axes:
        axr.grid(alpha=.15)
        axr.legend(loc="upper right",ncol=2)
        axr.set_ylabel("amplitude")
    axes[-1].set_xlabel("coordinate")
    fig.tight_layout()
    save_figure(fig,out,"13_recovered_packets")

    # 14 Reconstruction residual.
    reconstructed = recovered_background.values(x)+recovered_edge.values(x)
    residual = reconstructed-signal.values(x)
    fig,ax=plt.subplots(figsize=(12.2,4.5))
    ax.plot(x,residual,color=RED,lw=1.4)
    ax.axhline(0,color=INK,lw=.8)
```

```
    ax.set_xlabel("coordinate")
    ax.set_ylabel("residual")
    ax.set_title(f"Complete reconstruction residual, maximum {np.max(np.abs(residual)):.2e}",weight="bold")
    ax.grid(alpha=.16)
    save_figure(fig,out,"14_exact_reconstruction_residual")

    # 15 Raw Fourier versus resolved synthesis.
    order = background.degree + 12
    all_measurements = {n: signal.fourier(n) for n in range(-order,order+1)}
    raw = raw_fourier_partial(all_measurements,x,order)
    resolved = recovered_background.values(x)+recovered_edge.values(x)
    fig,axes=plt.subplots(2,1,figsize=(12.5,6.7),sharex=True)
    axes[0].plot(x,signal.values(x),color=INK,lw=2,label="true")
    axes[0].plot(x,raw,color=BLUE,lw=1.1,label=f"raw Fourier order {order}")
    axes[0].set_title("Raw global truncation",loc="left",weight="bold")
    axes[1].plot(x,signal.values(x),color=INK,lw=2,label="true")
    axes[1].plot(x,resolved,color=RED,lw=1.1,ls="--",label="recovered edge + smooth packet")
    axes[1].set_title("Projector-resolved synthesis",loc="left",weight="bold")
    for axr in axes:
        axr.grid(alpha=.15)
        axr.legend(loc="upper right",ncol=2)
        axr.set_ylabel("amplitude")
    axes[-1].set_xlabel("coordinate")
    fig.suptitle("Unknown edges are first recovered, then synthesized in their native packet",fontsize=17,
     weight="bold")
    fig.tight_layout(rect=[0,0,1,.94])
    save_figure(fig,out,"15_gibbs_free_synthesis")

    # 16 Edge zoom.
    center=edge.locations[0]
    local=np.abs(((x-center+PI)%TAU)-PI)<.35
    fig,ax=plt.subplots(figsize=(11.5,4.8))
    ax.plot(x[local],signal.values(x)[local],color=INK,lw=2,label="true")
    ax.plot(x[local],raw[local],color=BLUE,lw=1.4,label="raw Fourier")
    ax.plot(x[local],resolved[local],color=RED,lw=1.2,ls="--",label="resolved")
    ax.axvline(center,color=ORANGE,lw=1,ls=":")
    ax.set_xlabel("coordinate")
    ax.set_ylabel("amplitude")
    ax.set_title("Local view: the reconstructed edge has no Fourier ringing",weight="bold")
    ax.grid(alpha=.16)
    ax.legend(loc="best")
    save_figure(fig,out,"16_edge_zoom")

    # 17 Exact cases dashboard.
    fig,axes=plt.subplots(1,3,figsize=(13.0,4.5))
    Ls=[int(r["true_edges"]) for r in exact_rows]
    axes[0].semilogy(Ls,[float(r["location_error"]) for r in exact_rows],marker="o",color=BLUE)
    axes[0].set_title("location error",weight="bold")
    axes[1].semilogy(Ls,[float(r["jump_error"]) for r in exact_rows],marker="o",color=ORANGE)
    axes[1].set_title("jump error",weight="bold")
    axes[2].semilogy(Ls,[float(r["reconstruction_error"]) for r in exact_rows],marker="o",color=RED)
    axes[2].set_title("reconstruction error",weight="bold")
    for axr in axes:
        axr.set_xlabel("number of edges")
        axr.grid(alpha=.17,which="both")
    fig.suptitle("Exact off-grid recovery across several unknown-edge counts",fontsize=16.5,weight="bold")
    fig.tight_layout(rect=[0,0,1,.92])
    save_figure(fig,out,"17_exact_case_dashboard")

    # 18 Finite dictionary theorem.
    problem,y,alpha,recovered,gamma=dictionary_data
    n=np.arange(y.size)
    fig,axes=plt.subplots(3,1,figsize=(12.3,7.0),sharex=True)
    axes[0].plot(n,y,color=INK,lw=2)
    axes[0].set_title("observed discrete signal",loc="left",weight="bold")
    axes[1].stem(n,problem.dictionary@alpha,linefmt=ORANGE,markerfmt="o",basefmt=" ")
    axes[1].set_title("true two-atom edge packet",loc="left",weight="bold")
    axes[2].stem(n,recovered["edge"],linefmt=RED,markerfmt="x",basefmt=" ")
    axes[2].set_title(f"recovered packet, restricted separation $\\gamma_4={gamma:.3f}$",loc="left",weight="
     bold")
    for axr in axes:
```

```
        axr.grid(alpha=.15)
        axr.set_ylabel("value")
    axes[-1].set_xlabel("sample index")
    fig.tight_layout()
    save_figure(fig,out,"18_finite_dictionary_recovery")

    # 19 Secant geometry.
    fig,ax=diagram_axes((12.5,5.6))
    draw_box(ax,(.06,.43),.20,.20,"two candidate\nedge packets",face=ORANGE+"20",edge=ORANGE,fontsize=15)
    draw_box(ax,(.39,.43),.22,.20,"subtract\n" + r"$D(\alpha-\beta)$",face=WHITE,edge=INK,fontsize=14)
    draw_box(ax,(.74,.58),.20,.16,"visible after\n" + r"$R_{\mathcal{S}}$",face=TEAL+"20",edge=TEAL,fontsize
     =13)
    draw_box(ax,(.74,.25),.20,.16,"lost inside the\nsmooth packet",face=RED+"18",edge=RED,fontsize=13)
    draw_arrow(ax,(.26,.53),(.39,.53))
    draw_arrow(ax,(.61,.55),(.74,.66),color=TEAL,text=r"$RDh\ne0$",text_offset=(0,.04))
    draw_arrow(ax,(.61,.49),(.74,.33),color=RED,text=r"$RDh=0$",text_offset=(0,-.04))
    ax.text(.5,.10,r"Universal $s$-sparse recovery holds exactly when no nonzero $2s$-sparse secant is hidden
      by the background.",ha="center",fontsize=13.5)
    save_figure(fig,out,"19_residualized_secant_condition")

    # 20 Noise by separation heatmap.
    separations=sorted({r["separation"] for r in noise_rows})
    sigmas=sorted({r["sigma"] for r in noise_rows})
    matrix=np.zeros((len(separations),len(sigmas)))
    for row in noise_rows:
        matrix[separations.index(row["separation"]),sigmas.index(row["sigma"])]=row["success_rate"]
    fig,ax=plt.subplots(figsize=(11.5,5.3))
    im=ax.imshow(matrix,aspect="auto",origin="lower",vmin=0,vmax=1,cmap="viridis")
    ax.set_xticks(range(len(sigmas)),[f"{s:.0e}" for s in sigmas],rotation=35,ha="right")
    ax.set_yticks(range(len(separations)),[f"{s:.2f}" for s in separations])
    ax.set_xlabel("Fourier coefficient noise standard deviation")
    ax.set_ylabel("minimum edge separation")
    ax.set_title("Exact-location success in the controlled noisy experiment",weight="bold")
    for i in range(len(separations)):
        for j in range(len(sigmas)):
            val=matrix[i,j]
            ax.text(j,i,f"{val:.2f}",ha="center",va="center",color=WHITE if val<.6 else INK,fontsize=9)
    fig.colorbar(im,ax=ax,fraction=.035,label="success rate")
    fig.tight_layout()
    save_figure(fig,out,"20_noise_separation_heatmap")

    # 21 Noise location error.
    fig,ax=plt.subplots(figsize=(11.5,5.3))
    for sep,color in zip(separations,PALETTE,strict=False):
        rows=[r for r in noise_rows if r["separation"]==sep]
        ax.loglog([max(r["sigma"],1e-12) for r in rows],[max(r["median_location_error"],1e-14) for r in rows],
     marker="o",lw=2,color=color,label=f"sep {sep:.2f}")
    ax.set_xlabel("noise standard deviation")
    ax.set_ylabel("median location error")
    ax.set_title("Edge localization is stable only while the Hankel and root packets remain separated",weight
     ="bold")
    ax.grid(alpha=.18,which="both")
    ax.legend(loc="upper left",ncol=2)
    save_figure(fig,out,"21_noise_location_error")

    # 22 Close-edge conditioning.
    fig,axes=plt.subplots(1,2,figsize=(12.8,4.9))
    d=[r["minimum_separation"] for r in close_rows]
    axes[0].loglog(d,[r["hankel_condition"] for r in close_rows],marker="o",color=BLUE)
    axes[0].set_title("Hankel condition number",weight="bold")
    axes[1].loglog(d,[r["vandermonde_condition"] for r in close_rows],marker="o",color=MAGENTA)
    axes[1].set_title("Vandermonde condition number",weight="bold")
    for axr in axes:
        axr.set_xlabel("minimum edge separation")
        axr.grid(alpha=.18,which="both")
        axr.invert_xaxis()
    fig.suptitle("There is no uniform stable recovery theorem as distinct edges collide",fontsize=16.5,weight
     ="bold")
    fig.tight_layout(rect=[0,0,1,.92])
    save_figure(fig,out,"22_collision_conditioning")

```

```
    # 23 Collision moment decay.
    deltas=np.geomspace(.8,.002,100)
    nidx=np.arange(1,13)
    norms=[np.linalg.norm(1-np.exp(-1j*nidx*d)) for d in deltas]
    fig,ax=plt.subplots(figsize=(10.8,5.0))
    ax.loglog(deltas,norms,color=RED,lw=2)
    ax.set_xlabel("distance between opposite jumps")
    ax.set_ylabel("norm of measured moment vector")
    ax.set_title("Opposite jumps become invisible as their separation tends to zero",weight="bold")
    ax.grid(alpha=.18,which="both")
    save_figure(fig,out,"23_no_uniform_stability")

    # 24 Multi-channel.
    multi_result:RecoveryResult=multichannel["result"] # type: ignore[assignment]
    loc=np.asarray(multichannel["locations"])
    fig,axes=plt.subplots(2,1,figsize=(11.8,5.8),sharex=True)
    axes[0].bar(np.arange(loc.size),np.asarray(multichannel["jumps1"]),color=TEAL)
    axes[0].set_title("channel 1 jump amplitudes",loc="left",weight="bold")
    axes[1].bar(np.arange(loc.size),np.asarray(multichannel["jumps2"]),color=ORANGE)
    axes[1].set_title("channel 2 jump amplitudes",loc="left",weight="bold")
    axes[1].set_xticks(np.arange(loc.size),[f"edge {i+1}" for i in range(loc.size)])
    for axr in axes:
        axr.axhline(0,color=INK,lw=.8)
        axr.grid(alpha=.15,axis="y")
    fig.suptitle(f"Shared edge locations are recovered jointly, maximum error {location_error(loc,
     multi_result.locations):.2e}",fontsize=16.2,weight="bold")
    fig.tight_layout(rect=[0,0,1,.92])
    save_figure(fig,out,"24_multichannel_shared_edges")

    # 25 Piecewise-polynomial repeated roots.
    fig,ax=diagram_axes((12.8,5.8))
    draw_box(ax,(.04,.43),.19,.20,"piecewise polynomial\n$u$ of degree $d$",face=BLUE+"20",edge=BLUE,fontsize
     =14)
    draw_box(ax,(.30,.43),.20,.20,"distributional derivative\n$D^{d+1}u$",face=TEAL+"20",edge=TEAL,fontsize
     =14)
    draw_box(ax,(.57,.43),.18,.20,"exponential-polynomial\nmoments",face=ORANGE+"20",edge=ORANGE,fontsize=13)
    draw_box(ax,(.82,.43),.14,.20,"repeated-root\nannihilator",face=MAGENTA+"20",edge=MAGENTA,fontsize=13)
    draw_arrow(ax,(.23,.53),(.30,.53))
    draw_arrow(ax,(.50,.53),(.57,.53))
    draw_arrow(ax,(.75,.53),(.82,.53))
    ax.text(.5,.17,r"Each edge location becomes a root of multiplicity at most $d+1$ in the annihilating
     polynomial.",ha="center",fontsize=14)
    ax.text(.5,.10,"The piecewise-constant theorem is the case d = 0.",ha="center",color=MUTED,fontsize=11)
    save_figure(fig,out,"25_piecewise_polynomial_extension")

    # 26 Root perturbation disks.
    roots=np.exp(-1j*edge.locations)
    fig,ax=plt.subplots(figsize=(6.5,6.5))
    ax.add_patch(plt.Circle((0,0),1,fill=False,color=GRID,lw=2))
    radius=min(.18,edge.min_separation/5)
    for i,z in enumerate(roots):
        ax.add_patch(plt.Circle((z.real,z.imag),radius,fill=False,color=PALETTE[i%len(PALETTE)],lw=1.5))
        ax.scatter([z.real],[z.imag],color=PALETTE[i%len(PALETTE)],s=70)
    ax.set_aspect("equal")
    ax.set_xlim(-1.3,1.3)
    ax.set_ylim(-1.3,1.3)
    ax.axhline(0,color=GRID,lw=.8)
    ax.axvline(0,color=GRID,lw=.8)
    ax.set_title("A computable Rouche certificate isolates one recovered root in each disk",weight="bold")
    save_figure(fig,out,"26_root_stability_disks")

    # 27 Runtime.
    fig,ax=plt.subplots(figsize=(10.8,5.0))
    ax.plot([r["edge_order"] for r in runtime_rows],[1e3*r["seconds"] for r in runtime_rows],marker="o",color
     =PURPLE,lw=2)
    ax.set_xlabel("number of edges")
    ax.set_ylabel("milliseconds per recovery")
    ax.set_title("Reference Prony implementation cost",weight="bold")
    ax.grid(alpha=.18)
    save_figure(fig,out,"27_runtime_scaling")

```

```
    # 28 Measurement count.
    L=np.arange(1,13)
    fig,ax=plt.subplots(figsize=(10.5,5.0))
    ax.plot(L,2*L,marker="o",color=BLUE,lw=2,label="high-frequency edge moments")
    ax.plot(L,2*L+2*4+1,marker="o",color=ORANGE,lw=2,label="including degree-4 low band")
    ax.set_xlabel("number of unknown jumps")
    ax.set_ylabel("complex Fourier coefficients")
    ax.set_title("Finite-rate measurement count grows linearly with the number of edges",weight="bold")
    ax.grid(alpha=.18)
    ax.legend(loc="upper left")
    save_figure(fig,out,"28_measurement_count")

    # 29 Failure by too few moments.
    fig,ax=diagram_axes((12.6,5.4))
    draw_box(ax,(.08,.46),.22,.18,"L unknown edges\n2L degrees in the moment model",face=ORANGE+"20",edge=
     ORANGE,fontsize=14)
    draw_box(ax,(.41,.61),.22,.16,"2L consecutive moments\nfull Hankel factorization",face=TEAL+"20",edge=
     TEAL,fontsize=13)
    draw_box(ax,(.41,.26),.22,.16,"fewer moments\nnonunique annihilators",face=RED+"20",edge=RED,fontsize=13)
    draw_box(ax,(.76,.46),.16,.18,"exact roots\nor ambiguity",face=MAGENTA+"20",edge=MAGENTA,fontsize=14)
    draw_arrow(ax,(.30,.55),(.41,.69),color=TEAL)
    draw_arrow(ax,(.30,.55),(.41,.34),color=RED)
    draw_arrow(ax,(.63,.69),(.76,.59),color=TEAL)
    draw_arrow(ax,(.63,.34),(.76,.50),color=RED)
    ax.text(.5,.10,"The theorem uses enough moments to determine the annihilator rather than guessing a
     preferred edge set.",ha="center",fontsize=12.5)
    save_figure(fig,out,"29_too_few_moments")

    # 30 Unknown bandlimit failure.
    fig,ax=plt.subplots(figsize=(11.5,5.0))
    n=np.arange(1,19)
    edge_m=np.abs(edge.moments(n))
    full_m=np.abs(np.asarray([2j*PI*k*signal.fourier(int(k)) for k in n]))
    ax.semilogy(n,full_m,marker="o",color=BLUE,label="moments formed from full signal")
    ax.semilogy(n,edge_m,marker="o",color=ORANGE,label="true edge moments")
    ax.axvspan(1,background.degree,color=RED,alpha=.12,label="contaminated by smooth modes")
    ax.axvline(background.degree+.5,color=INK,ls="--")
    ax.set_xlabel("moment index")
    ax.set_ylabel("magnitude")
    ax.set_title("The high-frequency start must lie beyond a valid background bandlimit",weight="bold")
    ax.grid(alpha=.18,which="both")
    ax.legend(loc="upper right")
    save_figure(fig,out,"30_bandlimit_boundary")

    # 31 Tests dashboard.
    fig,ax=plt.subplots(figsize=(12.0,7.0))
    ax.axis("off")
    ax.text(.5,.96,"Independent checks in the single-file implementation",ha="center",fontsize=20,weight="
     bold",transform=ax.transAxes)
    y=.90
    for i,test in enumerate(tests):
        bg=PAPER if i%2==0 else "#EEF2F3"
        ax.add_patch(FancyBboxPatch((.04,y-.025),.92,.042,boxstyle="round,pad=.004,rounding_size=.004",
     facecolor=bg,edgecolor="none",transform=ax.transAxes))
        ax.text(.06,y,test["name"],ha="left",va="center",fontsize=9.5,transform=ax.transAxes)
        ax.text(.93,y,"PASS" if test["passed"] else "FAIL",ha="right",va="center",fontsize=9.5,weight="bold",
     color=TEAL if test["passed"] else RED,transform=ax.transAxes)
        y-=.047
    save_figure(fig,out,"31_verification_dashboard")

    # 32 Proof map.
    fig,ax=diagram_axes((13.0,6.2))
    nodes=[
        (.04,.66,.17,.16,"no-free-lunch\nboundary",RED),
        (.25,.66,.17,.16,"residualized\nedge packet",TEAL),
        (.46,.66,.17,.16,"finite dictionary\nsecant theorem",ORANGE),
        (.67,.66,.17,.16,"Fourier moment\nreduction",BLUE),
        (.82,.39,.14,.16,"Hankel rank\nand roots",MAGENTA),
        (.59,.20,.18,.16,"jumps and native\nedge synthesis",PURPLE),
        (.34,.20,.18,.16,"smooth packet\nreconstruction",GREEN),
        (.09,.20,.18,.16,"Gibbs-free\ncomplete signal",RED),
```

```
    ]
    for x0,y0,w,h,label,color in nodes:
        draw_box(ax,(x0,y0),w,h,label,face=color+"20",edge=color,fontsize=12)
    arrows=[((.21,.74),(.25,.74)),((.42,.74),(.46,.74)),((.63,.74),(.67,.74)),((.84,.66),(.87,.55))
     ,((.82,.39),(.77,.31)),((.59,.28),(.52,.28)),((.34,.28),(.27,.28))]
    for s,e in arrows:draw_arrow(ax,s,e)
    ax.text(.5,.93,"Proof dependency map",ha="center",fontsize=22,weight="bold")
    ax.text(.5,.08,"The exact theorem and the impossibility theorem belong together. The assumptions are part
      of the result.",ha="center",fontsize=12,color=MUTED)
    save_figure(fig,out,"32_proof_dependency_map")

    # 33 Status boundary.
    categories=["arbitrary functions\nfinite samples","finite dictionary\nsparse edges","1D off-grid\nfinite-
     rate edges","piecewise polynomial\nfinite innovation","arbitrary 2D\nunknown curves"]
    status=[0,2,2,1,0]
    labels=["impossible","proved","proved","reduced","not proved"]
    colors=[RED,TEAL,TEAL,ORANGE,RED]
    fig,ax=plt.subplots(figsize=(12.2,4.9))
    ax.bar(np.arange(len(categories)),[1]*len(categories),color=[c+"CC" for c in colors],edgecolor=INK)
    ax.set_xticks(np.arange(len(categories)),categories)
    ax.set_yticks([])
    for i,label in enumerate(labels):ax.text(i,.5,label,ha="center",va="center",color=WHITE,weight="bold",
     fontsize=12)
    ax.set_ylim(0,1)
    ax.set_title("Exact scope of the universal unknown-edge result",weight="bold")
    ax.spines[["left","right","top","bottom"]].set_visible(False)
    fig.tight_layout()
    save_figure(fig,out,"33_claim_boundary")

    # 34 Summary wheel.
    fig,ax=plt.subplots(figsize=(7.8,7.8))
    ax.set_aspect("equal")
    ax.axis("off")
    labels_w=[("model",BLUE),("project",TEAL),("differentiate",GREEN),("measure",YELLOW),("annihilate",ORANGE
    ),("locate",RED),("synthesize",MAGENTA),("check",PURPLE)]
    theta=np.linspace(90,90-360,len(labels_w)+1)
    from matplotlib.patches import Wedge
    for i,(label,color) in enumerate(labels_w):
        wedge=Wedge((0,0),1.0,theta[i+1],theta[i],width=.34,facecolor=color,edgecolor=PAPER,lw=2)
        ax.add_patch(wedge)
        angle=np.deg2rad((theta[i]+theta[i+1])/2)
        ax.text(.80*np.cos(angle),.80*np.sin(angle),label,ha="center",va="center",rotation=(theta[i]+theta[i
     +1])/2-90,rotation_mode="anchor",fontsize=12,weight="bold",color=WHITE)
    ax.add_patch(Circle((0,0),.48,facecolor=PAPER,edgecolor=INK,lw=2))
    ax.text(0,.08,"unknown-edge",ha="center",fontsize=16,weight="bold")
    ax.text(0,-.03,"recovery",ha="center",fontsize=21,weight="bold")
    ax.text(0,-.17,"exact within the declared class",ha="center",fontsize=10,color=MUTED)
    ax.set_xlim(-1.12,1.12)
    ax.set_ylim(-1.12,1.12)
    ax.set_title("The closed recovery cycle",fontsize=20,weight="bold",pad=12)
    save_figure(fig,out,"34_summary_wheel")


    # 35 Two-dimensional finite dictionary example.
    fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.8))
    panels = [
        rectangle_2d["observed"],
        rectangle_2d["edge"],
        rectangle_2d["recovered_edge"],
        np.abs(rectangle_2d["residual"]),
    ]
    titles = ["observed image", "true rectangle packet", "recovered packet", "absolute residual"]
    cmaps = ["viridis", "magma", "magma", "magma"]
    for axr, panel, title, cmap in zip(axes, panels, titles, cmaps, strict=True):
        image = axr.imshow(panel, origin="lower", cmap=cmap)
        axr.set_title(title, weight="bold", fontsize=10.5)
        axr.set_xticks([])
        axr.set_yticks([])
        fig.colorbar(image, ax=axr, fraction=.046)
    fig.suptitle(
        f"The finite dictionary theorem is dimension-independent: recovered rectangle {rectangle_2d['
```

```
        best_metadata']}",
            fontsize=15.5,
            weight="bold",
        )
        fig.tight_layout(rect=[0, 0, 1, .90])
        save_figure(fig, out, "35_two_dimensional_rectangle")


def write_csv(path: Path, rows: Sequence[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows:
        path.write_text("", encoding="utf-8")
        return
    fields = list(rows[0].keys())
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def run_all(base: Path) -> dict[str, object]:
    base.mkdir(parents=True, exist_ok=True)
    (base / "figures").mkdir(exist_ok=True)
    (base / "tables").mkdir(exist_ok=True)
    (base / "results").mkdir(exist_ok=True)

    exact_rows, representative = run_exact_examples()
    noise_rows = run_noise_experiment()
    close_rows = run_close_edge_experiment()
    runtime_rows = run_runtime_experiment()
    dictionary_data = build_dictionary_example()
    multichannel = build_multichannel_example()
    rectangle_2d = build_2d_rectangle_example()
    tests = run_tests(exact_rows, representative, dictionary_data, multichannel, rectangle_2d)
    generate_figures(base, exact_rows, representative, noise_rows, close_rows, runtime_rows, dictionary_data,
      multichannel, rectangle_2d, tests)

    write_csv(base / "tables" / "exact_recovery_cases.csv", exact_rows)
    write_csv(base / "tables" / "noise_and_separation.csv", noise_rows)
    write_csv(base / "tables" / "collision_conditioning.csv", close_rows)
    write_csv(base / "tables" / "runtime_scaling.csv", runtime_rows)
    write_csv(base / "tables" / "test_ledger.csv", tests)

    symbolic = exact_symbolic_example()
    result: RecoveryResult = representative["result"]  # type: ignore[assignment]
    edge: EdgeModel = representative["edge"]  # type: ignore[assignment]
    match = match_edge_models(edge, result)
    problem, y, alpha, recovered, gamma = dictionary_data
    summary: dict[str, object] = {
        "formal_scope": "periodic bandlimited background plus finitely many off-grid piecewise-constant jumps
    ",
        "exact_random_cases": len(exact_rows),
        "maximum_exact_location_error": max(float(r["location_error"]) for r in exact_rows),
        "maximum_exact_jump_error": max(float(r["jump_error"]) for r in exact_rows),
        "maximum_exact_reconstruction_error": max(float(r["reconstruction_error"]) for r in exact_rows),
        "representative_edge_order": edge.order,
        "representative_location_error": float(match["location_error_max"]),
        "representative_jump_error": float(match["jump_error_max"]),
        "representative_hankel_condition": result.hankel_condition,
        "representative_vandermonde_condition": result.vandermonde_condition,
        "finite_dictionary_gamma_4": gamma,
        "finite_dictionary_support": list(recovered["support"]),
        "rectangle_2d_true": list(rectangle_2d["true_metadata"]),
```

```
        "rectangle_2d_recovered": list(rectangle_2d["best_metadata"]),
        "symbolic_example": symbolic,
        "tests_passed": sum(bool(t["passed"]) for t in tests),
        "tests_total": len(tests),
        "figure_pairs": len(list((base / "figures").glob("*.pdf"))),
    }
    (base / "results" / "summary.json").write_text(json.dumps(summary, indent=2, sort_keys=True) + "\n",
     encoding="utf-8")
    return summary


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--out", type=Path, default=Path(__file__).resolve().parent)
    args = parser.parse_args()
    summary = run_all(args.out)
    print(json.dumps(summary, indent=2, sort_keys=True))


if __name__ == "__main__":
    main()
```

D Bibliographic note

Bibliography

[1] M. Vetterli, P. Marziliano, and T. Blu, “Sampling Signals with Finite Rate of Innovation,” *IEEE Transactions on Signal Processing*, vol. 50, no. 6, pp. 1417–1428, 2002. DOI: 10.1109/TSP.2002.1003065.

[2] P. L. Dragotti, M. Vetterli, and T. Blu, “Sampling Moments and Reconstructing Signals of Finite Rate of Innovation: Shannon Meets Strang–Fix,” *IEEE Transactions on Signal Processing*, vol. 55, no. 5, pp. 1741–1757, 2007. DOI: 10.1109/TSP.2006.890907.

[3] E. J. Candès and C. Fernandez-Granda, “Towards a Mathematical Theory of Super-Resolution,” *Communications on Pure and Applied Mathematics*, vol. 67, no. 6, pp. 906–956, 2014.

[4] G. Ongie and M. Jacob, “Off-the-Grid Recovery of Piecewise Constant Images from Few Fourier Samples,” *SIAM Journal on Imaging Sciences*, vol. 9, no. 3, pp. 1004–1041, 2016. DOI: 10.1137/15M1042280.

E Reproduction

From the source directory, run

```
python universal_unknown_edge_recovery.py --out .
pdflatex -interaction=nonstopmode Universal_Unknown_Edge_Recovery.tex
pdflatex -interaction=nonstopmode Universal_Unknown_Edge_Recovery.tex
```

The Python file writes every figure, table, and numerical result before the manuscript is compiled.